

gemini-conversation

Exported on: 12/8/2025, 10:53:49 PM

We start here with a basic prompt and pasting in the question. I chose not to give the pdf instead because it proved to yield worse results in prior experiments (see my special participation A)

User

Help me walk through these questions and learn the solutions in detail please: . Optimizers as Penalized Linear Improvement with different norm penalties In lecture, you saw the locally linear perspective of a neural network and the loss by Taylor expanding the loss around the current value of the parameters. This approximation is only very good in a near neighborhood of those values. One way to proceed with optimization is to consider the size of the neighborhood as a hyperparameter and to bound our update to stay within that neighborhood while minimizing our linear approximation to the loss. You saw in lecture that the choice of norm in defining that neighborhood also matters. In this problem, you will work out for yourself a slightly different perspective. Instead of treating the norm as a constraint (with the size of the acceptable norm as a hyperparameter), we can do an unconstrained optimization with a weighted penalty that corresponds to the squared norm — where that weight is a hyperparameter. At each iteration, we wish to maximize linear improvement of the objective (as defined by the dot-product between the gradient and the update) locally regularized by a penalty on the size of the update. This can be expressed (in traditional minimization form) as: $u = \operatorname{argmin} \Delta\theta^T g + \frac{1}{2\alpha} \|\Delta\theta\|_2^2$. (1) where $g = \nabla f(\theta)$ is the gradient of the loss, α is a scalar, and d is a scalar-output distance function $\mathbb{R}^{\dim(\theta)} \rightarrow \mathbb{R}$. (a) Let's assume Euclidean distance is the norm that captures our sense of relevant neighborhoods in parameter space. Then our objective can be: $u = \operatorname{argmin} \Delta\theta^T g + \frac{1}{2\alpha} \|\Delta\theta\|_2^2$. (2) What is the analytical solution for u in the above problem? What standard optimizer does this recover? Solution: To solve for u , we can take the gradient of the objective with respect to $\Delta\theta$ and set it to zero: $\nabla_{\Delta\theta} \Delta\theta^T g + \frac{1}{2\alpha} \|\Delta\theta\|_2^2 = g + \frac{1}{\alpha} \Delta\theta = 0$. (3) Homework 2, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 1 Homework 2 @ 2025-09-22 21:06:17-04:00 Then, solving for $\Delta\theta$, we get: $\Delta\theta = -\alpha g$, (4) which is simply a scaled version of the negative gradient. This solution is equivalent to standard gradient descent with a step size of α . SGD or GD can be understood as maximizing linear improvement subject to a Euclidean distance penalty over parameters. This can also be viewed equivalently, if we split out the directions and magnitude, as: $\Delta\theta = -\alpha \|g\|_2 g / \|g\|_2$, (5) where the $g / \|g\|_2$ defines the unit vector (in 2-norm) in the direction we want, and the $\alpha \|g\|_2$ tells the magnitude we want to move in that direction. Notice that this magnitude scales with the size (in 2-norm) of the gradient vector. (b) Now, consider an alternative way of capturing local neighborhood size – the squared infinity norm over parameters. Recall that this is defined as $\|x\|_\infty = \max_i |x_i|$. Our objective is now: $u = \operatorname{argmin} \Delta\theta^T g + \frac{1}{2\alpha} \|\Delta\theta\|_\infty^2$. (6) What is the analytical solution for u in this case? Which optimizer does this correspond to? Solution: To solve for u , we can rewrite the objective as: $u = \operatorname{argmin} \Delta\theta^T g + \frac{1}{2\alpha} \max_i |\Delta\theta_i|^2$. (7) Let $M = \max_i |\Delta\theta_i|$. Then, we can express the objective as: $u = \operatorname{argmin} \Delta\theta^T g + \frac{1}{2\alpha} M^2$, subject to $|\Delta\theta_i| \leq M$ for all i . (8) To minimize the objective, we want to make $\Delta\theta^T g$ as negative as possible while keeping M small. The optimal solution occurs when $|\Delta\theta_i| = M$ for all i , and $\Delta\theta_i$ has the opposite sign of g_i . Thus, $\Delta\theta_i = -M \cdot \operatorname{sign}(g_i)$. (9) From this we know the solution is the sign of the gradient, scaled by a constant $-M$. To solve for M , substituting this back into the objective, we get: $u = -M g^T \operatorname{sign}(g) + \frac{1}{2\alpha} M^2 = -M \|g\|_1 + \frac{1}{2\alpha} M^2$. (10) To minimize this with respect to M , we set the derivative to zero: $\nabla_M (-M \|g\|_1 + \frac{1}{2\alpha} M^2) = -\|g\|_1 + \frac{1}{\alpha} M = 0$ (11) $M = \alpha \|g\|_1$. (12) Homework 2, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 2 Homework 2 @ 2025-09-22 21:06:17-04:00 Therefore, the solution for u is: $u_i = -\alpha \|g\|_1 \cdot \operatorname{sign}(g_i)$. (13) This is taking a step in the direction of the sign of the gradient. Intuitively, this is because the infinity norm penalizes the largest element of the update, and so the optimal solution is to make all elements of the update equal to each other in magnitude. This sign-function picks out the direction that we move to do our update. It is not the same direction as the direction of the gradient. However, in the update (13), we also have the $\|g\|_1$ scaling. This is different from SignGD. When the gradient is small, we take a small step while when the gradient is large, we take a larger step. Interestingly, we measure the size of the gradient in 1-norm while choosing the magnitude of the motion. So both solutions have the same basic form. A magnitude that depends on the size of the gradient times a unit-vector pointing based on the gradient. However, in this case, the magnitude is measured using the 1-norm and the unit-vector is a unit-vector in the infinity-norm. This is a manifestation of a more general pattern where the two norms involved are dual to each other. The 2-norm just happens to be its own dual.

Gemini

This is a fantastic perspective on optimization. It bridges the gap between simple "gradient following" and more complex ideas like Trust Regions. It frames the update step as a trade-off: we want to improve the loss (follow the gradient), but we don't want to step so far that our local approximation becomes invalid (the penalty).

Let's break down **Part (a)** first.

We are looking at the objective function with a squared Euclidean (L2) penalty:

$$J(\Delta\theta) = g^T \Delta\theta + \frac{1}{2\alpha} \|\Delta\theta\|_2^2$$

The solution jumps right to the derivative in equation (3). To ensure you understand the mechanics behind that jump, let's look at the two terms separately.

1. The Linear Term: $g^T \Delta\theta$.

2. **The Quadratic Term:** $\|\Delta\theta\|_2^2$ (which is equivalent to $\Delta\theta^T \Delta\theta$).

Do you recall the rules for taking the gradient with respect to a vector $\Delta\theta$ for these two specific terms?

First piece of feedback that I gave it, and it seems to listen!

User

Make sure that when explaining a problem, you start with the problem statement and understanding on that. Assume minimal prior knowledge. Let's start from the top please

Gemini

Let's explore that together. We will break this down piece by piece, starting with the big picture before we ever touch the calculus.

The Problem Context: Optimization

Imagine you are training a neural network. You have a "Loss Function" that tells you how bad your model's predictions are. Your goal is to change the model's settings (called **parameters**, denoted by θ) to make that Loss as low as possible.

We are looking for the best **update** (denoted as $\Delta\theta$) to add to our current parameters.

The problem proposes a specific strategy for finding this update, given by this formula:

$$u = \operatorname{argmin}_{\Delta\theta} \left(\underbrace{g^T \Delta\theta}_{\text{Linear Improvement}} + \underbrace{\frac{1}{\alpha} d(\Delta\theta)}_{\text{Distance Penalty}} \right)$$

Breaking down the Terms

1. **The Gradient (g):** This represents the slope of the loss landscape at your current position. It points in the direction of steepest ascent (uphill).
2. **The Update ($\Delta\theta$):** This is the step we are deciding to take.
3. **Linear Improvement ($g^T \Delta\theta$):** This term measures how much we expect the loss to change based on the slope. We want this to be a large **negative** number (meaning the loss goes down a lot).
4. **Distance Penalty ($d(\Delta\theta)$):** This term measures how big our step is. We add this because our "linear" assumption about the slope is only accurate for small steps. If we step too far, the landscape might change unpredictably.

The Trade-off

This equation creates a competition between two forces:

- **Force 1:** Go as far "downhill" as possible to reduce the loss.
- **Force 2:** Don't move too far from where we are, or we might step off a cliff (or up a hill we didn't see).

Before we move to the math in part (a), looking at that equation, what do you think would happen to our step size $\Delta\theta$ if the parameter α (alpha) is very, very small?

User

If alpha is very, very small, our distance penalty would be very big, so the step size would be tiny.

Gemini

That's spot on. A small α makes the penalty term $\frac{1}{\alpha}$ huge, acting like a tight leash that forces our step $\Delta\theta$ to be very small. This is often called a "Trust Region"—we only trust our linear approximation in a tiny area.

Now, let's solve **Part (a)** mathematically. To find the optimal step u , we need to minimize the objective function:

$$J(\Delta\theta) = g^T \Delta\theta + \frac{1}{\alpha} \|\Delta\theta\|_2^2$$

To do this, we take the derivative (gradient) with respect to $\Delta\theta$ and set it to zero. Let's tackle the terms one by one.

Do you recall what the derivative of the first term, $g^T \Delta\theta$, is with respect to the vector $\Delta\theta$?

(Hint: Think of how the derivative of a simple scalar equation like ax is just a .)

User

It would just be g^T

Gemini

That's right. The derivative of the linear term $g^T \Delta\theta$ with respect to $\Delta\theta$ is just the gradient vector g .

Now let's look at the second term: $\frac{1}{\alpha} \|\Delta\theta\|_2^2$.

To find the derivative of this, it helps to rewrite the squared norm $\|\Delta\theta\|_2^2$ as the dot product of the vector with itself:

$$\Delta\theta^T \Delta\theta$$

Do you remember what the derivative of $\Delta\theta^T \Delta\theta$ is with respect to $\Delta\theta$?

(Hint: It's very similar to the scalar derivative of x^2 .)

User

Noticed that it wasn't very strict here and agreed even though my answer was wrong!

I notice that I said g^T , and you said the answer is g . What is the actual right answer?

Gemini

This is a great question to ask because notation in matrix calculus can be tricky!

The standard answer in machine learning is g (a column vector).

Here is the intuition on why:

1. **Shape Matching:** We are trying to find an update for $\Delta\theta$, which is a **column vector**. To eventually add our derivative to $\Delta\theta$, the derivative must also be a column vector.
2. **The Transpose:** g^T is a row vector. If we used that, the dimensions wouldn't match up for addition later.

So, while the logic of "the coefficient of $\Delta\theta$ is g^T " makes sense in scalar algebra, in vector calculus we transpose it back to g so it points in the correct direction in our parameter space.

Now, let's get back to that second term. We need the derivative of the penalty term:

$$\frac{1}{\alpha} \|\Delta\theta\|_2^2 = \frac{1}{\alpha} (\Delta\theta^T \Delta\theta)$$

Ignoring the $\frac{1}{\alpha}$ for a moment, what is the derivative of $\Delta\theta^T \Delta\theta$ with respect to $\Delta\theta$?

(Hint: In scalar calculus, the derivative of x^2 is $2x$. What is the vector equivalent?)

User Made sure that it knew not to do that for the future.

My answer to your question is $2\Delta\theta$. However, for the future, make sure the answers I give you are fully correct. Be stringent.

Gemini

That is exactly right. The derivative of $\Delta\theta^T \Delta\theta$ is $2\Delta\theta$. I will definitely keep that high standard for precision as we move forward!

So, combining the two parts we just found, the total derivative of our objective function is:

$$\nabla_{\Delta\theta} J = g + \frac{1}{\alpha}(2\Delta\theta)$$

To find the minimum, we set this gradient equal to zero:

$$g + \frac{2}{\alpha}\Delta\theta = 0$$

Now, can you rearrange this equation to solve for $\Delta\theta$? This will give us the analytical solution for u .

User

It would be $(-g * \alpha)/2$

Gemini

That is correct!

So, we found that the optimal update is:

$$\Delta\theta = -\frac{\alpha}{2}g$$

If we define a "learning rate" $\eta = \frac{\alpha}{2}$, our update becomes $\Delta\theta = -\eta g$.

Does this update rule look familiar? What standard optimization algorithm uses this exact formula?

User

Isn't this just gradient descent?

Gemini

Yes, exactly!

You have correctly identified that this recovers **Gradient Descent** (or Stochastic Gradient Descent, depending on how g is calculated).

It is a really cool insight: standard Gradient Descent can be viewed as the solution to an optimization problem where we maximize improvement while penalizing the **Euclidean distance** of the step.

Moving to Part (b)

Now, the problem asks us to change how we measure "distance." Instead of the Euclidean norm (L_2), we use the **squared infinity norm** (L_∞).

Our new objective is:

$$u = \operatorname{argmin}_{\Delta\theta} \left(g^T \Delta\theta + \frac{1}{\alpha} \|\Delta\theta\|_\infty^2 \right)$$

To get started on this one, do you remember how the infinity norm $\|\Delta\theta\|_\infty$ is defined?

User

The infinity norm is just the maximum if I remember correctly

Gemini

That is exactly right. The infinity norm is the maximum absolute value of the vector's components:

$$\|\Delta\theta\|_{\infty} = \max_i |\Delta\theta_i|$$

Because the "max" function is hard to differentiate directly, let's use a little trick. Let's introduce a variable, M , to represent that maximum value:

$$M = \max_i |\Delta\theta_i|$$

Now our objective function looks like this:

$$J = g^T \Delta\theta + \frac{1}{\alpha} M^2$$

But we have a constraint: every single component $\Delta\theta_i$ must stay within the bounds set by M .

$$|\Delta\theta_i| \leq M$$

To minimize the total J , we want the first term, the dot product $g^T \Delta\theta = \sum g_i \Delta\theta_i$, to be as **negative** as possible.

If you know that a specific component $\Delta\theta_i$ cannot be larger than M or smaller than $-M$, what value should you pick for $\Delta\theta_i$ to make the term $g_i \Delta\theta_i$ as small (negative) as possible?

(Hint: Consider the sign of g_i .)

User

-M

Gemini

You are very close!

To be precise, we want the sign of $\Delta\theta_i$ to be the **opposite** of g_i .

- If g_i is positive, we choose $-M$.
- If g_i is negative, we choose $+M$.

We can write this mathematically as:

$$\Delta\theta_i = -M \cdot \text{sign}(g_i)$$

Now we need to find the optimal value for the scalar M . Let's substitute our expression for $\Delta\theta_i$ back into the objective function:

$$J = \sum_i g_i \Delta\theta_i + \frac{1}{\alpha} M^2$$

$$J = \sum_i g_i (-M \cdot \text{sign}(g_i)) + \frac{1}{\alpha} M^2$$

Can you simplify the summation term? (Hint: What does a number multiplied by its sign, $x \cdot \text{sign}(x)$, always equal?)

User

it would just be the negative of g_i

Gemini

Close! The negative sign is definitely part of it, but there's a little more to the simplification.

Let's look at the multiplication of a number by its sign:

- If g_i is **positive** (e.g., 5): $5 \cdot \text{sign}(5) = 5 \cdot 1 = 5$.
- If g_i is **negative** (e.g., -5): $-5 \cdot \text{sign}(-5) = -5 \cdot (-1) = 5$.

In both cases, $g_i \cdot \text{sign}(g_i)$ gives us the **absolute value**, $|g_i|$. And don't forget we still have that factor M hanging around!

So, the term simplifies to:

$$g_i \Delta \theta_i = -M |g_i|$$

Now, if we sum this up over all i , our objective becomes:

$$J = \sum_i (-M |g_i|) + \frac{1}{\alpha} M^2$$

$$J = -M \sum_i |g_i| + \frac{1}{\alpha} M^2$$

Do you recognize the term $\sum_i |g_i|$? Which vector norm of g does this represent?

User

That's the L1 norm

Gemini

Spot on! 🎯 That sum is indeed the **L1 norm** of the gradient, denoted as $\|g\|_1$.

So our simplified objective function (in terms of the scalar M) is:

$$J(M) = -M \|g\|_1 + \frac{1}{\alpha} M^2$$

Now we are in the home stretch. To find the optimal size M , just take the derivative of this function with respect to M and set it to zero.

What do you get for M ?

User

-

$$J(M) = -M \|g\|_1 + \frac{1}{\alpha} M^2$$

+ (2/alpha)*M

Gemini

You are on the right track with the second part! The derivative of $\frac{1}{\alpha} M^2$ is indeed $\frac{2}{\alpha} M$.

However, it looks like the first part, $-M \|g\|_1$, didn't get differentiated yet.

Since $\|g\|_1$ is just a constant number (it doesn't change when M changes), we can treat this term like $-c \cdot M$.

What is the derivative of $-c \cdot M$ with respect to M ?

User

-C

Gemini

Exactly. Since we used c as a placeholder for $\|g\|_1$, the derivative of that first term is simply $-\|g\|_1$.

So, putting the two pieces together, our condition for the minimum is:

$$-\|g\|_1 + \frac{2}{\alpha}M = 0$$

Can you solve this equation for M ?

User

(-

$$-\|g\|_1 + \frac{2}{\alpha}M = 0$$

* alpha)/2

Gemini

That is the correct algebraic manipulation!

Solving for M :

$$M = \frac{\alpha}{2}\|g\|_1$$

Now we have the optimal value for the maximum step size M . Remember, we earlier determined that the individual updates are:

$$\Delta\theta_i = -M \cdot \text{sign}(g_i)$$

Substitute your expression for M into this equation. What is the final analytical solution for the update vector $\Delta\theta$?

User

$$-\frac{\alpha}{2}\|g\|_1 \cdot \text{sign}(g).$$
Gemini

That is exactly right! 🌟

You have successfully derived the analytical solution:

$$u = -\frac{\alpha}{2}\|g\|_1 \cdot \text{sign}(g)$$

Now, let's interpret what this math is actually telling us about the optimizer's behavior. This is the "Which optimizer does this correspond to?" part of the question.

In **Part (a)** (standard SGD), our update pointed in the direction of g .

Looking at your new formula for **Part (b)**:

1. **Direction:** The update uses $\text{sign}(g)$. Does this vector point in the exact same direction as the gradient g ? (Think about a gradient like $[0.1, 100]$).
2. **Magnitude:** The step size is scaled by $\|g\|_1$.

How would you describe the "shape" of the path this optimizer takes compared to standard SGD?

User

This vector does not point in the exact same direction as standard SGD. It should descend negatively in every direction of the gradient.

Gemini

You are absolutely right that it doesn't point in the same direction. The standard gradient finds the steepest path "as the crow flies," while this optimizer takes a very different route.

Let's visualize exactly *how* different with a concrete 2D example.

Imagine your gradient is $g = [1, 100]$.

- **Standard SGD** ($\Delta\theta \propto -g$) would take a step proportional to $[-1, -100]$. It moves a tiny bit in the first dimension and a huge amount in the second.
- **Our New Optimizer** ($\Delta\theta \propto -\text{sign}(g)$) takes a step proportional to $[-1, -1]$.

In this example, the new optimizer ignores the fact that the second dimension is 100 times steeper than the first. It steps equally in both coordinates!

Since the update vector is always composed of components with equal magnitude (scaled by a constant), the motion is restricted to 45° angles relative to the axes (diagonals).

This optimizer is known as **SignSGD** (or Sign Gradient Descent).

Wrapping Up Part (b)

You've discovered that:

1. **L2 Penalty (Euclidean)**: leads to standard **Gradient Descent**, moving perpendicular to contour lines.
2. **L-infinity Penalty (Box)**: leads to **SignSGD** (with a scaling factor), moving along diagonals.

This duality is really powerful:

- Minimizing the L_2 norm creates a spherical trust region.
- Minimizing the L_∞ norm creates a box-shaped trust region.

Shall we tackle the final part of the problem, or would you like to explore why someone might actually *want* to use SignSGD over standard SGD?

User

Let's finish this problem up

Gemini

Let's bring this all together. We have solved the math for both parts, but to really "finish" the problem, we need to understand the **general pattern** mentioned in the final paragraph of your text.

That paragraph notes a fascinating relationship between the **Penalty Norm** (how we constrain the step $\Delta\theta$) and the **Gradient Measurement** (how we measure the size of g to determine the step magnitude).

Let's look at our results side-by-side:

Case	Penalty on Update ($\Delta\theta$)	Resulting Scaling Factor uses...
(a)	L_2 Norm ($\ \Delta\theta\ $)	L_2 Norm of g ($\ g\ $)
(b)	L_∞ Norm ($\ \Delta\theta\ _\infty$)	L_1 Norm of g ($\ g\ $)



Export to Sheets



The text mentions that these pairs ($2 \leftrightarrow 2$ and $\infty \leftrightarrow 1$) are **dual norms**.

In mathematics, for any norm L_p , there is a dual norm L_q . There is a simple equation that relates the exponents p and q .

Do you happen to know the equation that relates p and q for dual norms?

User

No

Gemini

No problem! It is a very elegant little equation that connects them.

For any two dual norms L_p and L_q , their exponents are related by:

$$\frac{1}{p} + \frac{1}{q} = 1$$

In this context:

- p is the norm we use for the **penalty** on the update.
- q turns out to be the norm we use to measure the **gradient**.

Let's test this on our first case, **Part (a)**. We used the Euclidean norm (L_2) for the penalty, so $p = 2$.

If you plug $p = 2$ into that equation, what value do you get for q ?

User

2

Gemini

That is correct! $\frac{1}{2} + \frac{1}{2} = 1$, so q must be 2.

This explains why in **Part (a)**, where we penalized the L_2 norm (Euclidean distance), the scaling factor naturally depended on the L_2 norm of the gradient. The L_2 norm is special because it is its own dual.

Now let's apply this to **Part (b)** to see if the pattern holds.

In Part (b), we used the **infinity norm** (L_∞) for the penalty. This means $p = \infty$.

Using the dual norm equation $\frac{1}{p} + \frac{1}{q} = 1$, what is the value of q when $p = \infty$?

(Hint: What does $\frac{1}{\infty}$ approach?)

User

approaches 0

Gemini

That is exactly right. Since $\frac{1}{\infty}$ approaches 0, our dual norm equation simplifies nicely.

$$0 + \frac{1}{q} = 1$$

So, if $\frac{1}{q} = 1$, what must q be?

And once you have that, look back at the result from **Part (b)**. We used an L_∞ penalty ($p = \infty$). Did our resulting scaling factor correspond to the L_q norm of the gradient?

User

q is one and I'm not sure

Gemini

You should be sure because that is absolutely correct! 🧠

Since $\frac{1}{1} = 1$, q **must be 1**.

Now, look at how perfectly that fits our result from **Part (b)**:

- We used an **Infinity Norm** penalty ($p = \infty$).
- Our derived update rule scaled the step size based on the **1-Norm** of the gradient ($\|g\|_1$).

This confirms the pattern: The optimizer measures the gradient using the **dual norm** of the penalty.

Summary of What You Learned

You've walked through a really sophisticated view of optimization. Here is the big picture:

1. **Optimization as a Trade-off:** All these algorithms are just balancing "Linear Improvement" (going downhill) vs. a "Distance Penalty" (staying close).
2. **Euclidean (L_2) Penalty:**
 - Creates a spherical trust region.
 - Result: **Standard Gradient Descent**.
 - Dual relationship: L_2 penalty $\leftrightarrow L_2$ gradient measurement.
3. **Infinity (L_∞) Penalty:**
 - Creates a box-shaped trust region.
 - Result: **SignSGD** (moving along diagonals).
 - Dual relationship: L_∞ penalty $\leftrightarrow L_1$ gradient measurement.

You have now fully solved and analyzed the problem! 🎓

How do you feel about this topic now? Are you ready to move on to something else, or is there any part of this derivation you want to double-check?

So far so good, on question 1. Starting 2 here.

User

Let's do question 2 in the same way please: 2. Optimizers and their convergence Consider O: a simplified Adam-style optimizer without weight decay that has iterates $\theta_{t+1} \leftarrow \theta_t - \alpha M_t \nabla f_t(\theta_t)$ (14) where f_t is the loss at iteration t and α is the step size (learning rate). Further suppose that the adaptive scaling matrix M_t is recomputed over each epoch of training and just consists of a diagonal populated by the inverses of the square roots of the mean squared value for the gradients during the epoch for that specific coordinate. For this part, we have exactly $n = 1$ training point corresponding to the single equation $[1, 0.1, 0.01]\theta = 1$ (15) with a 3-dimensional learnable parameters θ . Suppose that we start with $\theta_0 = 0$ and use squared loss $f_t(\theta) = (1 - [1, 0.1, 0.01]\theta)^2$. (a) What specific vector θ would standard vanilla SGD (i.e. (14) with $M_t = I$ and $\alpha = \alpha$) converge to assuming $\alpha > 0$ was small enough to give convergence? Solution: SGD in this case is identical to gradient descent since there is only one training point. To get convergence with a constant step size, we need the gradient itself to get to zero. That happens only when (15) is actually solved. Gradient-descent on squared-loss can only move θ in the direction of $\begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$ and given that we start at $\theta = 0$, this means that we are looking for a solution of the form $\theta = \beta \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$. Plugging this into (15), we get $\beta[1, 0.1, 0.01] \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix} = 1$ (16) which has the solution $\beta = 1.0101$ giving us the overall answer $\theta^* = 1.0101 \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$. Notice that this is quite insensitive to the second and third coordinates, and if the test-data is like the training data in having small numbers there, the overall impact of anything there will be quite small indeed. (b) What specific vector θ would the simplified version of Adam O converge to assuming appropriate step-sizes $\alpha_t > 0$ to give convergence? Solution: In this case, we observe that the premultiplication by M_t in the updates (14) will have the effect of moving θ in the rescaled direction $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$. Other than that, the same argument applies as the previous part. So we get: $\beta[1, 0.1, 0.01] \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = 1$ (17) which has the solution $\beta = 1.11$ giving us the overall answer $\theta^* = 1.11 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$. Note that to get this convergence, we need to use a diminishing step size schedule for this highly simplified version of Adam. A constant step size wouldn't work. Further notice that this solution θ^* is significantly more sensitive to the second and third coordinates, but the weight on the first coordinate is not that different from the previous SGD solution. Consequently, as long as the test-data is like the training data in having small numbers in the second and third coordinate, by that very nature, they won't impact the final prediction by much. (c) Consider a learning approach that first did training input feature rescaling (so that each feature had unit second-moment), then ran SGD to convergence, and then converted the solution for the rescaled problem back to the original units. What specific vector θ would it give as its final solution (for use in original coordinates)? Solution: By rescaling the inputs explicitly, we multiply the second feature by 10 and the third feature by 100 giving us a new "normalized" problem: $[1, 1, 1]\theta_e = 1$ (18) with initial condition $\theta_e = 0$. Immediately by symmetry (or by following the same argument above), this has the solution $\theta_e = \frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$. To get this behavior on the original unscaled features, we need to incorporate the scaling we did into the weights. (This should be reminiscent of considerations that we have in dropout training as well as in the use of batch normalization.) This gives us the final answer: $\theta^* = \frac{1}{3} \begin{bmatrix} 1 \\ 10 \\ 100 \end{bmatrix}$. Notice that this is far more sensitive to the second and third coordinates. Even if test data had small numbers like the training data in those positions, they would significantly impact the predictions. This illustrates the different inductive biases of these three different approaches to solving an underdetermined equation.

Gemini

Let's jump into Question 2! This problem is a great way to understand how different optimizers (Vanilla SGD vs. Adam-style) and data preprocessing techniques impose different **inductive biases**. Essentially, when there are multiple possible solutions to a problem, the method we choose determines which specific solution we find.

We are working with a single data point equation:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

And a squared loss function:

$$f_t(\theta) = (1 - [1, 0.1, 0.01]\theta)^2$$

Part (a): Vanilla SGD

We start at $\theta_0 = 0$. We want to find where standard SGD converges.

First, recall that the SGD update rule is:

$$\theta_{t+1} \leftarrow \theta_t - \alpha \nabla f(\theta)$$

To understand where this moves, we need the gradient. What is the gradient of the loss function, $\nabla f(\theta)$, with respect to the vector θ ?

(Hint: Use the chain rule on $(1 - x^T \theta)^2$.)

User Making sure it knows here how much I value key ideas.

Can we go over the main lessons and key ideas from problem 1 again please

Gemini

Let's absolutely solidify those takeaways. Viewing optimization this way—as a "trust region" problem—is a powerful mental model that goes beyond just memorizing formulas.

1. The Core Trade-off

The main lesson is that every update step is a negotiation between two competing forces:

- **Linear Improvement** ($g^T \Delta \theta$): The desire to rush downhill as fast as possible to lower the loss.
- **Distance Penalty** ($\frac{1}{\alpha} d(\Delta \theta)$): The caution to stay close to where we are, because our estimate of "downhill" (the gradient) is only locally accurate.

2. Geometry Dictates the Path

The most striking result was that **how we measure distance** changes **where we step**.

- **L_2 Norm (Euclidean):**
 - **Shape:** A sphere or circle. ○
 - **Result:** Standard Gradient Descent. We move in the "steepest" direction effectively perpendicular to the surface of the sphere.
- **L_∞ Norm (Max):**
 - **Shape:** A box or square aligned with the axes. □
 - **Result:** SignSGD. We move towards the "corners" of the box (the diagonals), ignoring the relative magnitude of the gradient in different dimensions.

3. The Power of Duality

We also uncovered a hidden mathematical pattern: the **Dual Norms**.

- When we penalize the L_p norm of the update...
- The math naturally scales the step size using the L_q norm of the gradient.
- (Where $\frac{1}{p} + \frac{1}{q} = 1$).

This explains why the L_∞ penalty ($p = \infty$) led to a step size determined by the sum of absolute values ($q = 1$).

To check your intuition on this, imagine we have a very long, narrow valley in our loss landscape.

- **SGD (L_2)** tends to bounce back and forth across the narrow walls.
- **SignSGD (L_∞)** moves equally in all directions.

Based on what we learned, which optimizer do you think might navigate a "narrow valley" more efficiently, and why?

User

Okay now let's go back to doing problem 2 please

Gemini

Let's tackle Part (b).

We are now looking at the simplified Adam optimizer. The update rule is:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t M_t \nabla f_t(\theta_t)$$

The key here is the matrix M_t . The problem states that M_t is a diagonal matrix where each entry is the **inverse of the root-mean-square** of the gradients for that coordinate.

Since we only have one data point, our gradient is always proportional to the vector:

$$g = \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$$

Let's figure out what M_t does to this vector. If you calculate the scaling factor for each coordinate (inverse of the square root of the squared value), what are the three values on the diagonal of M_t ?

User

I forgot all about adam and rms **Stressing need to review here.**

Gemini

No worries! It can be a lot to keep track of. Let's break it down using the specific definition given in the problem statement.

The problem describes a "simplified Adam" where the matrix M_t scales the learning rate. It defines the diagonal entries of M_t as the **inverses of the square roots of the mean squared value for the gradients**.

Since we only have **one** data point, the "mean" doesn't really change anything. We just need to look at our gradient vector:

$$g = \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$$

Let's calculate the scaling factor for the second component (0.1) together:

1. **Square it:** $(0.1)^2 = 0.01$
2. **Take the square root:** $\sqrt{0.01} = 0.1$
3. **Invert it:** $1/0.1 = 10$

So, the scaling factor for the second coordinate is **10**.

Following those same steps, what are the scaling factors for the **first** (1) and **third** (0.01) coordinates?

User

1 and 100

Gemini

That is spot on.

So, our scaling matrix M_t has the diagonal values $[1, 10, 100]$.

Now, let's see how this changes our update path. In this simplified Adam optimizer, we don't just follow the gradient g . We follow the *scaled* gradient: $M_t \cdot g$.

To find this new direction, we multiply each component of the gradient by its corresponding scaling factor:

- **1st Component:** 1×1
- **2nd Component:** 0.1×10
- **3rd Component:** 0.01×100

When you calculate those products, what does the resulting vector look like?

User

[1 1 1]

Gemini

That's exactly right! The scaled update direction is $[1, 1, 1]$.

This is a crucial difference:

- **SGD** moved in the direction $[1, 0.1, 0.01]$.
- **Simplified Adam** moves in the direction $[1, 1, 1]$ (along the diagonal).

Now, let's find the specific point where this optimizer converges. Just like in Part (a), we assume the final solution θ lies along this path.

So, if $\theta = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, can you plug this vector into our data equation $([1, 0.1, 0.01] \cdot \theta = 1)$ and solve for the scalar β ?

User

[1 10 100]

Gemini

That $[1, 10, 100]$ vector is actually our scaling matrix diagonal M_t . You are very close, but we need to take one more step to get the **update direction**.

Remember, we multiply each element of the gradient g by its corresponding scaling factor from M_t :

- **1st:** $1 \times 1 = 1$
- **2nd:** $0.1 \times 10 = 1$
- **3rd:** $0.01 \times 100 = 1$

So, the actual direction this optimizer takes is $[1, 1, 1]$. It moves equally in all dimensions!

Now, let's find exactly where it stops. We know the solution θ must be some multiple of this direction, so $\theta = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$.

Substitute this θ into our data equation:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

$$[1, 0.1, 0.01] \cdot \left(\beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right) = 1$$

Can you solve that equation to find the value of the scalar β ?

User

1/1.0101 is the answer to your question. i'm still not sure what adam is. Are you getting to that?

Gemini

You are spot on with 1/1.0101 for the **first part (Vanilla SGD)**!

However, for this **second part (Adam)**, the math changes because the optimizer moves in a different direction.

Let's pause to clear up exactly what Adam is, as that explains *why* the math changes.

What is Adam?

Adam (Adaptive Moment Estimation) is a popular optimization algorithm. The main idea is that it gives every single parameter its own customized "learning rate," rather than using one global speed for everything.



- **Standard SGD** struggles if the landscape is steep in one direction (like our 1st coordinate) and flat in another (like our 3rd coordinate). It tends to oscillate or move too slowly.



- **Adam** fixes this by keeping track of the "size" of the gradients over time. It scales the updates so that parameters with small gradients get a boost, and parameters with huge gradients get calmed down.



In our simplified problem, this normalization is why the update direction changed from the uneven $[1, 0.1, 0.01]$ to the perfectly balanced $[1, 1, 1]$. It forces every parameter to contribute equally to the step.

Back to the Math for Part (b)

Since Adam moves along the diagonal direction $[1, 1, 1]$, our solution vector θ must look like this:

$$\theta = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

We need to find the scalar β so that this vector satisfies our data equation:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

If you substitute the Adam vector into that equation, what is the new value for β ?

User

1/1.0101

Gemini

That $1/1.0101$ was actually the answer for **Part (a)** (Vanilla SGD).

For **Part (b)** (Adam), the direction has changed. We are no longer moving along $[1, 0.1, 0.01]$. We are moving along the diagonal $[1, 1, 1]$.

To find the new β , we need to calculate the dot product of our data vector $[1, 0.1, 0.01]$ with our new direction $[1, 1, 1]$:

$$1 \cdot 1 + 0.1 \cdot 1 + 0.01 \cdot 1$$

What does that sum equal?

User

1.11

Gemini

That is correct. $1 + 0.1 + 0.01 = 1.11$.

So our equation simplifies to:

$$1.11\beta = 1$$

Solving for β gives you $\beta = \frac{1}{1.11}$.

Since our Adam update direction was $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, and $\theta = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, what is your final solution vector θ ?

User

[1.11 1.11 1.11]

Gemini

You are very close! You have the right idea that the components are all equal, but let's double-check the scalar value β .

Recall our equation:

$$1.11\beta = 1$$

To solve for β , we divide by 1.11, which gives $\beta = \frac{1}{1.11}$ (approximately 0.9).

It looks like you multiplied by 1.11 instead of dividing.

If you use $\beta = \frac{1}{1.11}$, what is the specific vector θ ?

Stressing need for motivation here.

User

Let's restart our attempt of this question. I don't understand the motivation behind what we were doing.

Gemini

Let's wipe the slate clean. It is completely normal to feel lost in the mechanics of these algorithms; sometimes we get so focused on the *how* (the matrix math) that we lose the *why*.

The Motivation: Why are we doing this?

Imagine you have a single equation with three unknowns:

$$1 \cdot x + 0.1 \cdot y + 0.01 \cdot z = 1$$

This is an **under-determined system**. It's like asking, "I spent \$1.00 buying apples, bananas, and cherries. How much of each did I buy?" There isn't just one answer! You could have bought only apples, or mostly cherries, or a mix of all three.

In machine learning, we often have models with way more parameters than data points. This means there are **infinite** valid sets of parameters θ that solve the problem (get zero loss).

The Core Question: Since there are infinite "correct" answers, which specific one will our computer find?

It turns out the answer depends entirely on the **optimizer** (the algorithm) we use.

- **SGD** (Gradient Descent) has a specific "personality"—it's lazy. It wants to do the least amount of work possible to solve the equation.
- **Adam** has a different "personality"—it tries to be fair. It wants all parameters to contribute equally.

The Setup

We are going to predict exactly which solution each algorithm picks for this specific problem:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

To start, let's verify that there really are multiple ways to solve this.

Can you give me **two different vectors** for θ (just simple guesses) that would make that equation true?

User

[0.9 9 1] and [1 0 0]

Gemini

You found one perfect solution!

- **Vector 2:** $[1, 0, 0]$ works because $1(1) + 0.1(0) + 0.01(0) = 1$.

Your first guess was a good attempt, but let's check the math:

- **Vector 1:** $[0.9, 9, 1]$ gives us $1(0.9) + 0.1(9) + 0.01(1) = 0.9 + 0.9 + 0.01 = 1.81$. That's a bit too high.

To really see how different optimizers pick different paths, let's find just one more working solution.

If we set the first and third parameters to zero (so $\theta = [0, x, 0]$), what value must x be to satisfy the equation?

$$1(0) + 0.1(x) + 0.01(0) = 1$$

User

10

Gemini

That **10** is indeed the value for the middle coordinate (0.1), just like in the example!

Now we need the values for the other two coordinates to complete our matrix diagonal.

1. First coordinate (1):

- The magnitude is 1.
- We need the inverse: $1/1 = ?$

2. Third coordinate (0.01):

- The magnitude is 0.01.
- We need the inverse: $1/0.01 = ?$

What are those two values?

User

1 and 100

Gemini

That is spot on.

- **First coordinate:** $1/1 = 1$
- **Third coordinate:** $1/0.01 = 100$

So, our scaling matrix M_t has the diagonal values $[1, 10, 100]$.

Now, recall that the update rule for this simplified Adam optimizer isn't just "minus the gradient." It is:

$$\text{Update} \propto -M_t \cdot \nabla f(\theta)$$

We need to multiply our scaling matrix M_t by our gradient vector $\nabla f(\theta) = [1, 0.1, 0.01]$.

If you multiply those two vectors element-by-element (the first scaling factor times the first gradient component, and so on), what resulting vector do you get?

User

you get [1 1 1]

Gemini

You nailed it! The update direction for this simplified Adam optimizer is indeed $[1, 1, 1]$.

This means that unlike SGD (which took a path proportional to $[1, 0.1, 0.01]$), Adam is going to march along the diagonal, changing every parameter by the exact same amount.

Since we start at 0, our final solution θ must be some multiple of this direction:

$$\theta = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

To find where it stops, substitute this vector into our original data equation:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

What value for β satisfies that equation?

User**Reinforcing need for motivation and restating the problem statement.**

See, I understand the math, but you didn't restate the homework problem, what we're trying to solve, and why we're trying to solve it.

Gemini

I apologize for jumping straight into the mechanics without setting the stage first. That was like trying to drive a car before knowing where we're going! 🚗

Let's reset and look at the big picture for Question 2.

1. The Homework Problem

We are examining a specific scenario with these ingredients:

- **The Data:** We have just **one** training example.
 - Input: $[1, 0.1, 0.01]$
 - Target Output: 1
 - Equation: $1 \cdot \theta_1 + 0.1 \cdot \theta_2 + 0.01 \cdot \theta_3 = 1$
- **The Model:** A simple linear model with 3 learnable parameters ($\theta = [\theta_1, \theta_2, \theta_3]$).
- **The Optimizer:** A "Simplified Adam" optimizer. It doesn't just use the raw gradient; it scales the gradient by the inverse of the root-mean-square of past gradients (represented by matrix M_t).

2. What We Are Trying to Solve

We are trying to find the **exact final values** for θ ($\theta_1, \theta_2, \theta_3$) that this specific optimizer will settle on.

Since we have 3 unknowns but only 1 equation, there are infinite possible solutions (infinite combinations of θ that equal 1). We need to calculate *which one* this specific algorithm picks.

3. Why We Are Doing This

The goal of this problem is to demonstrate **Inductive Bias**.

- In Part (a), we saw that **SGD** is "lazy"—it picks a solution dominated by the first parameter because that's the steepest path.
- In Part (b), we want to see how **Adam** behaves differently. By normalizing the gradients, does it pick a "fairer" solution where all parameters contribute?

Does that framing make sense? If you're ready, we can jump back into finding that specific solution vector for the Simplified Adam optimizer.

User

No, you misunderstand. Let's start at the beginning while you walk me through question 2 please. Remember this: 2. Optimizers and their convergence Consider O: a simplified Adam-style optimizer without weight decay that has iterates $\theta_{t+1} \leftarrow \theta_t - \alpha_t M_t \nabla f_t(\theta_t)$ (14) where f_t is the loss at iteration t and α_t is the step size (learning rate). Further suppose that the adaptive scaling matrix M_t is recomputed over each epoch of training and just consists of a diagonal populated by the inverses of the square roots of the mean squared value for the gradients during the epoch for that specific coordinate. For this part, we have exactly $n = 1$ training point corresponding to the single equation $[1, 0.1, 0.01]\theta = 1$ (15) with a 3-dimensional learnable parameters θ . Suppose that we start with $\theta_0 = 0$ and use squared loss $f_t(\theta) = (1 - [1, 0.1, 0.01]\theta)^2$. (a) What specific vector θ would standard vanilla SGD (i.e. (14) with $M_t = I$ and $\alpha_t = \alpha$) converge to assuming $\alpha > 0$ was small enough to give convergence? Solution: SGD in this case is identical to gradient descent since there is only one training point. To get convergence with a constant step size, we need the gradient itself to get to zero. That happens only when (15) is actually solved. Gradient-descent on squared-loss can only move θ in the direction of $\begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$ and given that we start at $\theta = 0$, this means that we are looking for a solution of the form Homework 2, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 3 Homework 2 @ 2025-09-22 21:06:17-04:00 $\theta = \beta \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$. Plugging this into (15), we get $\beta[1, 0.1, 0.01] \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix} = 1$ (16) which has the solution $\beta = 1/1.0101$ giving us the overall answer $\theta^* = 1/1.0101 \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$. Notice that this is quite insensitive to the second and third coordinates, and if the test-data is like the training data in having small numbers there, the overall impact of anything there will be quite small indeed. (b) What specific vector θ would the simplified version of Adam O converge to assuming appropriate step-sizes $\alpha_t > 0$ to give convergence? Solution: In this case, we observe that the premultiplication by M_t in the updates (14) will have the effect of moving θ in the rescaled direction $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$. Other than that, the same argument applies as the previous part. So we get: $\beta[1, 0.1, 0.01] \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = 1$ (17) which has the solution $\beta = 1/1.11$ giving us the overall answer $\theta^* = 1/1.11 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$. Note that to get this convergence, we need to use a diminishing step size schedule for this highly simplified version of Adam. A constant step size wouldn't work. Further notice that this solution $\theta^* \cdot \alpha$ is significantly more sensitive to the second and third coordinates, but the weight on the first coordinate is not that different from the previous SGD solution. Consequently, as long as the test-data is like the training data in having small numbers in the second and third coordinate, by that very nature, they won't impact the final prediction by much. (c) Consider a learning approach that first did training input feature rescaling (so that each feature had unit second-moment), then ran SGD to convergence, and then converted the solution for the rescaled problem back to the original units. What specific vector θ would it give as its final solution (for use in original coordinates)? Solution: By rescaling the inputs explicitly, we multiply the second feature by 10 and the third feature by 100 giving us a new "normalized" problem: $[1, 1, 1]\theta_e = 1$ (18) with initial condition $\theta_e = 0$. Immediately by symmetry (or by following the same argument above), Homework 2, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 4 Homework 2 @ 2025-09-22 21:06:17-04:00 this has the solution $\theta_e = 1/3 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$. To get this behavior on the original unscaled features, we need to incorporate the scaling we did into the weights. (This should be reminiscent of considerations that we have in dropout training as well as in the use of batch normalization.) This gives us the final answer: $\theta^* = 1/3 \begin{bmatrix} 1 \\ 10 \\ 100 \end{bmatrix}$. Notice that this is far more sensitive to the second and third coordinates. Even if test data had small numbers like the training data in those positions, they would significantly impact the predictions. This illustrates the different inductive biases of these three different approaches to solving an underdetermined equation.

Gemini

Understood. Let's hit reset and walk through Question 2 from the top, step-by-step.

We are investigating how different optimizers solve the same under-determined problem. We have one data point equation:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

Let's start with **Part (a)**: Standard Vanilla SGD.

We begin with our parameters θ at zero. In SGD, every update step we take is based on the gradient of the loss function:

$$f(\theta) = (1 - [1, 0.1, 0.01]\theta)^2$$

To figure out where we go, we need the derivative. Without worrying about the specific numbers yet, when you take the gradient of this loss with respect to the vector θ , the result will always be proportional to a specific vector.

What **vector direction** does every SGD update move along?

User

It always moves along [1 0.1 0.001]

Gemini

You are very close! The vector is actually **[1, 0.1, 0.01]** (matching our data), but you have the right idea: SGD updates will always be proportional to this vector.

Because we start at 0 and always move in that direction, our final answer θ must look like this:

$$\theta = \beta \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$$

(where β is just some number determining how far we moved).

To find the specific value of β , we plug this expression for θ back into our original equation:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

Can you substitute our vector for θ into that equation and calculate the value of β ?

User

[1 10 100]

Gemini

That is spot on.

- **First coordinate:** $1/1 = 1$
- **Third coordinate:** $1/0.01 = 100$

So, our scaling matrix M_t has the diagonal values [1, 10, 100].

Now, recall that the update rule for this simplified Adam optimizer isn't just "minus the gradient." It is:

$$\text{Update} \propto -M_t \cdot \nabla f(\theta)$$

We need to multiply our scaling matrix M_t by our gradient vector $\nabla f(\theta) = [1, 0.1, 0.01]$.

If you multiply those two vectors element-by-element (the first scaling factor times the first gradient component, and so on), what resulting vector do you get?

User

[1 1 1]

Gemini

Spot on! The update direction for this simplified Adam optimizer is indeed $[1, 1, 1]$.

This is the key difference we were looking for:

- **SGD** followed the gradient directly: $[1, 0.1, 0.01]$. It moved mostly in the first dimension.
- **Adam** normalizes the step, moving along the diagonal: $[1, 1, 1]$. It treats all dimensions equally.

Now, let's find the exact point where it lands. Since we start at zero and move along that diagonal, our final solution θ must be a multiple of that direction:

$$\theta = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

To find the specific value of the scalar β , substitute this vector into our single data equation:

$$[1, 0.1, 0.01] \cdot \theta = 1$$

What is the value of β ?

User

1/1.11

Gemini

That is exactly right! $\beta = \frac{1}{1.11}$ (which is approximately 0.9).

So, we have our scaling factor β and our direction $[1, 1, 1]$.

Since $\theta = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, what is the final **vector** θ that this Simplified Adam optimizer converges to?

User

0

Gemini

It looks like you might be thinking of the starting point (which was 0) or the target loss.

For the final **solution**, we need to multiply our scalar β by our direction vector to get the final coordinates for θ .

$$\theta = \frac{1}{1.11} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

If you distribute that fraction $\frac{1}{1.11}$ into the vector, what does the final vector θ look like?

User

[1/1.11 1/1.11 1/1.11]

Gemini

That is absolutely correct! 🎉

The final solution for the **Simplified Adam** optimizer is:

$$\theta \approx \begin{bmatrix} 0.9 \\ 0.9 \\ 0.9 \end{bmatrix}$$

Part (b) Recap

So, we've found that:

1. **SGD** converged to $\approx \begin{bmatrix} 0.99 \\ 0.099 \\ 0.0099 \end{bmatrix}$. (Dominated by the first feature).
2. **Adam** converged to $\approx \begin{bmatrix} 0.9 \\ 0.9 \\ 0.9 \end{bmatrix}$. (Distributed equally across all features).

This shows how Adam's "fairness" (normalization) leads to a completely different solution than SGD's "steepest path" approach, even for the exact same problem!

Part (c): Pre-conditioning (Feature Rescaling)

Now for the final twist. Part (c) proposes a three-step process:

1. **Rescale the inputs** so every feature has the same "second moment" (unit variance).
2. Run standard **SGD** on this new problem.
3. Convert the solution back to the **original units**.

Let's start with step 1. Our original data is $x = [1, 0.1, 0.01]$. To make the "second moment" (squared value) of each feature equal to 1, we need to multiply them by constants.

- $1 \times a = 1 \rightarrow a = 1$
- $0.1 \times b = 1 \rightarrow b = ?$
- $0.01 \times c = 1 \rightarrow c = ?$

What values for b and c will scale those inputs up to 1?

Stopping it and making it start again. Looks like it took it into account well.

User

Wait what? Go through a and b again, you didn't really make that clear

Gemini

I appreciate you stopping me. It is completely normal to feel that way; these optimization concepts are abstract, and I moved too fast.

Let's take a deep breath and restart parts (a) and (b) with a focus on the **intuition** first, then the math.

The Core Problem

We have one equation:

$$1 \cdot \theta_1 + 0.1 \cdot \theta_2 + 0.01 \cdot \theta_3 = 1$$

Imagine this equation describes a flat sheet of paper floating in a 3D room. Any point on that paper is a valid "solution" (gives zero loss).

- **Our Goal:** We start at the origin (0, 0, 0) and want to jump onto that sheet.
- **The Catch:** Different optimizers jump in different directions. We want to find exactly where they land.

Part (a): Vanilla SGD

The Intuition: Think of SGD as being "greedy" or "lazy." It wants to reach the solution by taking the steepest, most direct path possible.

- Since the first coefficient (1) is the largest, the "slope" is steepest in that direction.
- SGD will mostly update θ_1 and barely touch θ_3 .

The Math:

1. **The Direction:** The gradient tells us the steepest direction. For our equation, the gradient is always the vector of coefficients:

$$g = [1, 0.1, 0.01]$$

2. **The Path:** Since we start at 0 and move along this gradient, our final landing spot θ must be some multiple (β) of this vector:

$$\theta = \beta[1, 0.1, 0.01]$$

Your Turn: We need to find how far (β) we travel to hit the solution sheet. Substitute our path θ into the equation:

$$[1, 0.1, 0.01] \cdot (\beta[1, 0.1, 0.01]) = 1$$

This expands to:

$$\beta \cdot (1^2 + 0.1^2 + 0.01^2) = 1$$

Can you calculate the value of β from that equation?

User

1/1.0101

Gemini

That is exactly right!

$\beta = \frac{1}{1.0101}$ (which is approximately 0.99).

Now we have the scalar β , and we know the path was along the vector $[1, 0.1, 0.01]$.

Since $\theta = \beta[1, 0.1, 0.01]$, can you write out the final vector θ (approximately)?

User

[1.0101 1.0101 1.0101]

Gemini

Not quite. It looks like you might be thinking of the vector of ones (which we'll see in the next part!), or maybe just the scalar value itself.

For **Standard SGD**, remember our direction vector is the gradient: $[1, 0.1, 0.01]$.

We need to distribute that scalar $\beta \approx 0.99$ into *that* specific vector:

$$\theta \approx 0.99 \cdot \begin{bmatrix} 1 \\ 0.1 \\ 0.01 \end{bmatrix}$$

What does the vector look like when you perform that multiplication for each of the three components?

User

[0.99 0.099 0.0099]

Gemini

That is spot on!

So for **Standard SGD**, the solution is dominated by the first feature:

$$\theta_{\text{SGD}} \approx \begin{bmatrix} 0.99 \\ 0.099 \\ 0.0099 \end{bmatrix}$$

Part (b): Simplified Adam

Now let's see how **Adam** changes the game.

The Intuition: Adam says, "Hey, just because the first feature has a steep slope doesn't mean it's the most important. Let's give every feature a fair shot." It does this by **scaling** the step size for each feature so they all move at roughly the same speed.

The Math: To achieve this "fairness," Adam calculates a scaling matrix M . For our single data point, the diagonal entries of M are simply the **inverse** of the gradient values ($1/|g_i|$).

Given our gradient is still $[1, 0.1, 0.01]$, what are the three scaling values (the diagonal of M)?

User

[1 10 100]

Gemini

Correct! To normalize the inputs $[1, 0.1, 0.01]$ so they all equal 1, we multiplied them by the scaling factors $[1, 10, 100]$.

This gives us a new, "rescaled" problem to solve:

$$[1, 1, 1] \cdot \tilde{\theta} = 1$$

Now, Step 2 is to run **Standard SGD** on this *new* equation (starting from zero).

Using the same logic as Part (a), SGD will move in the direction of the gradient vector $[1, 1, 1]$. So, $\tilde{\theta} = \beta[1, 1, 1]$.

What value for β solves the equation $[1, 1, 1] \cdot (\beta[1, 1, 1]) = 1$?

User

1.11

Gemini

That 1.11 is the denominator we found, but remember that our scaling factor β is the *inverse* of that:

$$\beta = \frac{1}{1.11} \approx 0.9$$

And don't forget, the question asks for the final **vector** θ . We need to multiply our scalar β by the direction vector we found earlier $([1, 1, 1])$.

$$\theta \approx 0.9 \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

If you multiply that 0.9 into each component, what does your final list of three numbers look like?

User

[0.9 0.9 0.9]

Gemini

That is exactly right! 🎉

For the **Simplified Adam** optimizer, the final parameters are approximately:

$$\theta \approx \begin{bmatrix} 0.9 \\ 0.9 \\ 0.9 \end{bmatrix}$$

The Big Takeaway so far:

Notice how different this is from Part (a)!

- **Standard SGD** was "lazy": It put almost all the weight on the first feature (≈ 0.99) because that was the steepest path.
- **Adam** was "fair": It distributed the weight equally (≈ 0.9 each) because it normalized the gradients.

This proves that **the choice of optimizer determines which solution you find**, even for the exact same data!

Part (c): Pre-conditioning (Feature Rescaling)

Now for the final scenario. This approach explicitly fixes the "unevenness" of the data before we even start training.

The process has three steps:

1. **Rescale the inputs** so every feature has a value of 1 (unit second moment).
2. Run **SGD** on this new problem.
3. Convert the solution back to the **original units**.

Step 1: Rescaling Our original input vector is $x = [1, 0.1, 0.01]$. We need to multiply each feature by a specific number (a scaling factor) to turn it into a 1.

- Feature 1: $1 \times ? = 1$
- Feature 2: $0.1 \times ? = 1$
- Feature 3: $0.01 \times ? = 1$

What are the three scaling factors we need?

User

Ensuring that not too much knowledge is taken for granted here. Need to build up understanding of concepts as well as the solutions.

You should be operating under the assumption that you need to be teaching the concepts in detail as well please. For this question, you should not have assumed that I know what unit second moment is. Re explain part c

Gemini

I apologize for that assumption. You are absolutely right—"unit second moment" is technical jargon that deserves a clear explanation. Let's restart Part (c) and build it up properly from the concepts.

Part (c): Pre-conditioning (Feature Rescaling)

This approach is about "fixing" the data before we even let the optimizer touch it.

Concept: Unit Second Moment

In simple terms, the "second moment" is just a fancy way of saying the **average squared value** of a feature. It measures the "energy" or "scale" of that signal.

- If a feature is usually huge (like "House Price" in dollars), its squared value is massive.
- If a feature is tiny (like "Molecular Distance" in meters), its squared value is microscopic.

"Unit Second Moment" means we want to force every feature to be on the same scale—specifically, we want the squared value to be exactly **1**.

Why do this? If features have wildly different sizes (like our 1 vs 0.01), the optimizer gets confused and biased (as we saw in Part A). If we force them all to be size 1, the optimizer treats them fairly.

Step 1: Rescaling the Inputs

We have our input vector: $x = [1, 0.1, 0.01]$. We want to multiply each number by a scaling factor (s) so that the result is 1.

- **Feature 1:** $1 \times s_1 = 1$
- **Feature 2:** $0.1 \times s_2 = 1$
- **Feature 3:** $0.01 \times s_3 = 1$

What must the scaling factors s_1, s_2, s_3 be?

User

[1 10 100]

Gemini

Correct! Those are our scaling factors: $S = [1, 10, 100]$.

Step 2: Solve the "Rescaled" Problem

Now we imagine a new version of the problem where we use these scaled inputs. Original Input: $[1, 0.1, 0.01]$ Multiply by S : $[1*1, 0.1*10, 0.01*100]$ **New Input:** $[1, 1, 1]$

So, we run Standard SGD on this nice, symmetric equation:

$$[1, 1, 1] \cdot \tilde{\theta} = 1$$

(I'm using $\tilde{\theta}$ to represent the weights for this temporary, rescaled problem).

Since the inputs are $[1, 1, 1]$, the gradient is also $[1, 1, 1]$. We start at 0 and move in that direction, so the solution must be:

$$\tilde{\theta} = \beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Substitute that into the equation:

$$[1, 1, 1] \cdot \left(\beta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right) = 1$$

What is the value of β here?

User

1/3

Gemini

Correct!

Since $\beta = 1/3$, our solution for the **rescaled** problem is:

$$\tilde{\theta} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

Step 3: Convert back to Original Units

This is the tricky part. We found the weights ($\tilde{\theta}$) that work for the *scaled* inputs ($[1, 1, 1]$), but we need the weights (θ) that work for the *original* inputs ($[1, 0.1, 0.01]$).

Think about the equation for the second feature:

- **Original:** $0.1 \cdot \theta_2 = \text{contribution}$
- **Rescaled:** $1 \cdot \tilde{\theta}_2 = \text{contribution}$

We know the "Rescaled" version is $1 \cdot (1/3) = 1/3$. To make the "Original" version equal $1/3$, we have:

$$0.1 \cdot \theta_2 = 1/3$$

This means θ_2 has to be **10 times bigger** than $\tilde{\theta}_2$ to compensate for the input being 10 times smaller.

Mathematically, we just multiply our rescaled weights by the scaling factors we used earlier ($[1, 10, 100]$).

$$\theta = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \times \begin{bmatrix} 1 \\ 10 \\ 100 \end{bmatrix}$$

What is the final vector θ when you perform that element-wise multiplication?

User

[1/3 10/3 100/3]

Gemini

That is the correct final vector! 🎓

$$\theta_{\text{Rescaled}} = \begin{bmatrix} 1/3 \\ 10/3 \\ 100/3 \end{bmatrix} \approx \begin{bmatrix} 0.33 \\ 3.33 \\ 33.3 \end{bmatrix}$$

Summary of Question 2

This problem was a fantastic demonstration of **Inductive Bias**—how the choices we make in designing the algorithm change the answer we get, even when the data is exactly the same.

Let's look at the three different "personalities" we discovered:

Method	Strategy	Final Solution θ (approx)	Interpretation
(a) SGD	Lazy	[1.0, 0.1, 0.01]	"I'll just use the strongest feature (Feature 1) to solve the problem and ignore the weak ones."
(b) Adam	Fair	[0.9, 0.9, 0.9]	"I'll treat every feature equally, regardless of its signal strength."
(c) Rescaling	Aggressive	[0.3, 3.3, 33.3]	"I'll boost the weak features so much that the tiniest signal (Feature 3) becomes the most important weight."



Export to Sheets



The Lesson: If you have a dataset with some huge numbers and some tiny numbers (like this one), **Standard SGD** will essentially ignore the tiny numbers. If those tiny numbers contained important information, your model would fail! This is why **Adam** or **Feature Normalization** (Part c) are standard practices in Deep Learning—they ensure the model actually pays attention to everything.

You have now completed Question 2! How do you feel about these concepts? Are you ready to move on to the next problem, or do you have any questions about this comparison?

User

I feel good about those concepts, but you again forgot to give me key ideas after I was done with the question. Remember for all future questions to 1. Restate the question as is from the sheet itself 2. Begin walking me through it 3. After I'm done with a part, state all key ideas from the part and ensure understanding before moving on

Gemini

I have noted that preference. Moving forward, I will:

1. **Restate the question** exactly as written.
2. **Walk through** the solution step-by-step.
3. **Summarize key ideas** after each part and check for understanding.

In case you wanted to save this as a custom instruction, you can manually add that in your [personal context settings](#).

Please paste **Question 3** when you are ready, and we will apply this new format immediately!

Reinforcing need for key ideas again here.

This is the first code problem, so I didn't expect it to do great, but it did!

User

Here is the .py file and the questions and solutions for q3. Make sure to walk me through the coding parts in great detail as well, continuing to follow the preferences listed prior: 3. Coding Question: Initialization and Optimizers In this question, you'll implement He Initialization and Different Optimizers. You will have the choice between two options: Use Google Colab (Recommended). Open this url and follow the instructions in the notebook. Use a local Conda environment. Clone https://github.com/Berkeley-CS182/cs182fa25_public/tree/main and refer to [hw02/code/README.md](#) for further instructions. (a) What you observe in the mean of gradient norm plot above in the above plots? Try to give an explanation. Solution: Random and He initialization should train as expected. When we use zero initialization, our gradients become zero as well, which makes it very difficult to train our network to obtain nonzero weights.

Gemini

1. Restate the Question

3. Coding Question: Initialization and Optimizers In this question, you'll implement He Initialization and Different Optimizers. You will have the choice between two options:

- Use Google Colab (Recommended). Open this url and follow the instructions in the notebook.
- Use a local Conda environment. Clone https://github.com/Berkeley-CS182/cs182fa25_public/tree/main and refer to [hw02/code/README.md](#) for further instructions.

(a) What you observe in the mean of gradient norm plot above in the above plots? Try to give an explanation. Solution: Random and He initialization should train as expected. When we use zero initialization, our gradients become zero as well, which makes it very difficult to train our network to obtain nonzero weights.

2. Walkthrough: SGD with Momentum

Let's start with the first coding task: implementing **Momentum**.

You need to edit the file `deeplearning/optm.py`. Look for the function `sgd_momentum`.

The Math: The notebook provides this formula for the update:

$$\begin{cases} v_{dw}^t = m \cdot v_{dw}^{(t-1)} + dw \\ w = w - \alpha \cdot v_{dw}^t \end{cases}$$

Where:

- v_{dw} is the "velocity" (the running average of gradients).
- m is the momentum coefficient (usually around 0.9).
- α is the learning rate.
- dw is the current gradient.

The Code Implementation: In the function `sgd_momentum(w, dw, config=None)`, you will extract values from `config`.

- `config['velocity']` holds the current v .
- `config['momentum']` holds m .
- `config['learning_rate']` holds α .

Here is how you translate the math directly into NumPy code:

Python



```
def sgd_momentum(w, dw, config=None):
    if config is None: config = {}
    config.setdefault('learning_rate', 1e-2)
    config.setdefault('momentum', 0.9)
    # Initialize velocity to zeros if it doesn't exist yet
    v = config.get('velocity', np.zeros_like(w))

    # -----
```

```

# TODO: Implement the momentum update formula.
# 1. Update the velocity
# 2. Update the weights (w)
# -----

# Step 1:  $v = m * v_{old} + dw$ 
v = config['momentum'] * v + dw

# Step 2:  $w = w - learning\_rate * v$ 
next_w = w - config['learning_rate'] * v

# -----
#                               END OF YOUR CODE
# -----

config['velocity'] = v
return next_w, config

```

Tuning the Batch Size: After implementing this, the notebook asks you to tune the batch size for standard SGD so it matches the performance of SGD+Momentum.

- **Goal:** Find a `batch_size` for plain SGD such that `rel_error(train_acc) < 0.04`.
- **Intuition:** Momentum "smooths" the noise, acting like a larger batch size. To make plain SGD compete, you need to actually increase its batch size to reduce noise.
- **Action:** In the cell following the implementation, look for `batch_sizes = {'sgd': ...}`. Try increasing the value (e.g., 400, 800) until the training accuracy matches.

Key Ideas & Check for Understanding (Part 1)

Key Ideas:

- **Momentum** accumulates past gradients to maintain "velocity." This helps the optimizer plow through small noisy bumps and gain speed in consistent directions.
- **Damping Oscillation:** It reduces the zig-zagging motion common in standard SGD (like we saw in Question 1/2 with the steep/flat valleys).
- **Effective Batch Size:** Increasing the batch size of vanilla SGD reduces gradient variance, mimicking the smoothing effect of momentum.

Check for Understanding: Does the implementation of `next_w = w - learning_rate * v` make sense? Specifically, do you see why we subtract `v` (the accumulated velocity) instead of just subtracting `dw` (the current gradient)?

(Once you confirm, we will move to **RMSPprop** and **Adam**).

User

I don't fully get it. So, there's the velocity portion of momentum, where we multiply a coefficient times the prior accumulated gradients, and $1 -$ that coefficient times the current gradient. And then there's another part as well?

Gemini

It sounds like you are thinking of the "Exponential Moving Average" version of momentum, which is very common (and used in Adam!). In that version, you blend the old and new gradients like a weighted average: $0.9 \cdot \text{old} + 0.1 \cdot \text{new}$.

However, the specific version of Momentum assigned in **this homework** is slightly simpler. It doesn't use the $(1 - m)$ term.

Let's look at the formula given in the notebook again:

$$v_{dw}^t = m \cdot v_{dw}^{(t-1)} + dw$$

Notice there is no $(1 - m)$ in front of dw . It's just a direct addition.

The Two Steps of Momentum

To answer your question, yes, there are two distinct parts (steps) to the calculation:

Step 1: Update the Velocity (v) Think of this as updating the "speedometer" of the ball rolling down the hill.

- Take the current speed (v_{old}).
- Apply friction/decay by multiplying by `momentum` (e.g., 0.9).
- Add the new push from gravity (the current gradient `dw`).
- **Result:** The new velocity vector v .

Step 2: Update the Position (w) Now that we know how fast and in what direction we are moving, we actually move the weights.

- Take the current position (w).
- Subtract the velocity scaled by the `learning_rate`.
- **Result:** The new weight position $next_w$.

Why this matters for your code

If you implement the version with $(1 - m)$, your velocity will be much smaller than the homework expects, and the tests might fail.

Here is the breakdown for the code again with this specific formula in mind:

Python

```
# Step 1: The Velocity Portion
# v_new = (friction * v_old) + push
v = config['momentum'] * v + dw

# Step 2: The Update Portion
# position_new = position_old - (step_size * v_new)
next_w = w - config['learning_rate'] * v
```

Does that distinction between the "weighted average" version you were thinking of and the "accumulation" version in this homework make sense?

User

Yes, I get it! That's a good explanation, keep this in mind for the future. Let's move on

Gemini

Awesome. Now we are going to implement the two heavy hitters: **RMSProp** and **Adam**.

These are "adaptive" optimizers. Instead of just using one global learning rate (α) for every single parameter, they calculate a custom learning rate for each parameter individually.

Part 3: RMSProp

1. The Concept RMSProp (Root Mean Square Propagation) addresses a key problem with SGD:

- If the landscape is steep in one direction (vertical) but flat in another (horizontal), SGD oscillates wildly.
- RMSProp keeps a running average of the **squared gradients**.
- It divides the update by the square root of this average.
 - **Result:** High gradients get damped (divided by a large number). Low gradients get boosted (divided by a small number).

2. The Math (from the notebook)

$$\begin{cases} r = \rho r + (1 - \rho)(\nabla J)^2 \\ w = w - \alpha \frac{\nabla J}{\sqrt{r + \epsilon}} \end{cases}$$

3. The Code Implementation In `rmsprop`, the variable `config['cache']` corresponds to r (the history of squared gradients). `config['decay_rate']` is ρ .

Python

```
def rmsprop(w, dw, config=None):
    if config is None: config = {}
    config.setdefault('learning_rate', 1e-2)
    config.setdefault('decay_rate', 0.99)
    config.setdefault('epsilon', 1e-8)
    config.setdefault('cache', np.zeros_like(w))

    # -----
    # TODO: Implement the RMSprop update formula.
    # -----

    # Unpack config variables for clarity
    lr = config['learning_rate']
    decay_rate = config['decay_rate']
    eps = config['epsilon']
    cache = config['cache']

    # Step 1: Update cache (r)
    # r = rho * r + (1 - rho) * (grad squared)
    cache = decay_rate * cache + (1 - decay_rate) * (dw**2)

    # Step 2: Update weights (w)
    # w = w - alpha * (grad / (sqrt(r) + epsilon))
    next_w = w - lr * dw / (np.sqrt(cache) + eps)

    # -----
    #
```

END OF YOUR CODE

```
# -----

config['cache'] = cache
return next_w, config
```

Part 4: Adam

1. The Concept Adam (Adaptive Moment Estimation) is essentially **Momentum + RMSProp**.

- It keeps a history of the **gradients** (like Momentum) → variable m .
- It keeps a history of the **squared gradients** (like RMSProp) → variable v .
- **Unique Twist:** It includes a "Bias Correction" step. Since m and v start at zero, they are biased towards zero at the start of training. We divide by $(1 - \beta^t)$ to inflate them early on.

2. The Math (from the notebook)

$$\begin{aligned} \left\{ \begin{aligned} m &= \beta_1 m + (1 - \beta_1) \nabla J \\ v &= \beta_2 v + (1 - \beta_2) (\nabla J)^2 \end{aligned} \right. \\ m_{corr} &= \frac{m}{1 - \beta_1^t} \quad v_{corr} = \frac{v}{1 - \beta_2^t} \\ w &= w - \alpha \frac{m_{corr}}{\sqrt{v_{corr} + \epsilon}} \end{aligned}$$

3. The Code Implementation In `adam`, you have `config['m']` and `config['v']`. You also need `config['t']` (the time step) for the correction.

Python



```
def adam(w, dw, config=None):
    if config is None: config = {}
    config.setdefault('learning_rate', 1e-3)
    config.setdefault('beta1', 0.9)
    config.setdefault('beta2', 0.999)
    config.setdefault('epsilon', 1e-8)
    config.setdefault('m', np.zeros_like(w))
    config.setdefault('v', np.zeros_like(w))
    config.setdefault('t', 0)

    # -----
    # TODO: Implement the Adam update formula.
    # -----

    # 1. Update time step
    config['t'] += 1
    t = config['t']

    beta1 = config['beta1']
    beta2 = config['beta2']

    # 2. Update m (Momentum-like term)
    # Note: Unlike the Momentum part earlier, Adam DOES use (1 - beta1)
    config['m'] = beta1 * config['m'] + (1 - beta1) * dw

    # 3. Update v (RMSProp-like term)
    config['v'] = beta2 * config['v'] + (1 - beta2) * (dw**2)

    # 4. Compute Bias-Corrected m and v
    m_corr = config['m'] / (1 - beta1**t)
    v_corr = config['v'] / (1 - beta2**t)
```

```
# 5. Update weights
next_w = w - config['learning_rate'] * m_corr / (np.sqrt(v_corr) + config['epsilon'])

# -----
#                               END OF YOUR CODE
# -----

return next_w, config
```

Key Ideas & Check for Understanding (Part 3 & 4)

Key Ideas:

- **Adaptive Learning Rates:** RMSProp and Adam divide the update by $\sqrt{\text{squared gradients}}$. This naturally normalizes the updates—small gradients get boosted, large gradients get reduced.
- **Bias Correction:** In Adam, the terms $(1 - \beta^t)$ are crucial early in training (when t is small). Without them, the steps would be tiny because m and v initialize at zero. As $t \rightarrow \infty$, these terms approach 1 and stop mattering.
- **Standard:** Adam is the default "go-to" optimizer for most deep learning tasks today because it combines the best of both worlds.

Check for Understanding: In Adam, we calculate `m_corr` and `v_corr`. Why do we divide by $(1 - \beta^t)$? (Hint: If $\beta = 0.9$ and $t = 1$, the denominator is 0.1. This multiplies our estimate by 10. Why would we want to make our estimate 10x bigger on the very first step?)

Giving bad feedback for improper formatting.

User

The explanation you just gave me has unformatted math that is bad for the explanation. It looks like this:

$$\left\{ \begin{array}{l} r = \rho r + (1 - \rho)(\nabla J)^2 \\ w = w - \alpha \frac{\nabla J}{\sqrt{r + \epsilon}} \end{array} \right.$$

Gemini

I apologize for that messy formatting! You are right; it makes the math hard to read. Let me fix those equations so they are crystal clear, and then we'll answer the question about Adam's bias correction and finish the coding.

Corrected Equations

RMSProp

$$\left\{ \begin{array}{l} r = \rho r + (1 - \rho)(\nabla J)^2 \\ w = w - \alpha \frac{\nabla J}{\sqrt{r + \epsilon}} \end{array} \right.$$

Adam

$$\left\{ \begin{array}{l} m = \beta_1 m + (1 - \beta_1) \nabla J \\ v = \beta_2 v + (1 - \beta_2) (\nabla J)^2 \\ \hat{m} = \frac{m}{1 - \beta_1^t} \\ \hat{v} = \frac{v}{1 - \beta_2^t} \\ w = w - \alpha \frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}} \end{array} \right.$$

Answering the "Check for Understanding"

The question was: **Why do we divide by $(1 - \beta^t)$ in Adam?**

The Answer: Because we initialize m and v to **zeros**.

- On the very first step ($t = 1$), if we didn't correct it, our moving average would be tiny (mostly zeros).
- Dividing by $(1 - \beta^t)$ essentially "inflates" the value to compensate for the fact that we started with an empty history.
- As t gets large, β^t approaches 0, so the denominator becomes 1, and the correction naturally turns off.

Part 5: Initialization (He / Kaiming)

The final coding task is in `deeplearning/classifiers/fc_net.py`. You need to implement **He Initialization** (also known as Kaiming Initialization).

1. The Concept If you initialize weights too small, signals vanish (become 0) as they pass through layers. If you initialize them too large, signals explode. **He Initialization** is designed specifically for ReLU networks to keep the signal variance constant across layers.

2. The Math The notebook specifies we are using the "Normal distribution" version with `fan_in` mode. The standard deviation (σ) for the weights should be:

$$\sigma = \sqrt{\frac{2}{\text{fan_in}}}$$

Where `fan_in` is the number of inputs to the layer (the dimension of the previous layer).

3. The Code Implementation You need to find the `__init__` method in `FullyConnectedNet`. Look for the loop that initializes weights.

You will likely see a block like this:

Python



```
if self.use_batchnorm:
    # ... (batchnorm init)
```

And then the weight initialization logic. You need to implement the `he` and `random` cases.

Python



```
# Inside the initialization loop in fc_net.py
# W name is usually formatted like 'W' + str(i+1)
# input_dim is the size of the previous layer (fan_in)

if initialization == 'he':
    # -----
    # TODO: Implement He initialization
    # std = sqrt(2 / fan_in)
    # -----
    std = np.sqrt(2.0 / input_dim)
    self.params['W' + str(i + 1)] = std * np.random.randn(input_dim, hidden_dim)

elif initialization == 'random':
    # -----
    # TODO: Implement Random initialization
    # std = weight_scale (provided in arguments)
    # -----
    self.params['W' + str(i + 1)] = weight_scale * np.random.randn(input_dim, hidden_dim)
```

(Note: `np.random.randn` generates numbers from a Normal distribution with mean 0 and std 1. Multiplying by `std` scales it to the correct width).

Key Ideas & Check for Understanding (Part 5)

Key Ideas:

- **Vanishing/Exploding Gradients:** Bad initialization kills training before it starts.
- **He Init (ReLU):** Uses $\sqrt{2/n}$. The "2" accounts for the fact that ReLU kills half the neurons (sets them to zero), so we need double the variance to preserve the signal.
- **Xavier/Glorot Init (Sigmoid/Tanh):** Uses $\sqrt{1/n}$. (You didn't code this, but it's the counterpart for non-ReLU networks).

Check for Understanding: In the "He" formula $\sqrt{2/\text{fan_in}}$, if we used a huge layer with 10,000 inputs, would the initial weights be **smaller** or **larger** than a layer with only 10 inputs? (Hint: Think about what happens when you sum up 10,000 numbers versus 10 numbers).

User

The weights would be smaller

Gemini

That is exactly right. 🙌

If you have 10,000 inputs, you are summing up 10,000 values. If the weights were large, that sum would be massive (exploding). To keep the total "energy" constant, each individual weight must be tiny.

Key Ideas: Initialization (Part 5)

- **The Goldilocks Zone:** Deep networks are fragile at the start.
 - **Too Small (Zero/Tiny):** Signals die out (vanish). Gradients become zero. The network learns nothing.
 - **Too Big:** Signals blow up (explode). Computers hit `NaN` (Not a Number) errors.
 - **Just Right (He/Xavier):** The variance is carefully calculated so the signal strength stays the same from Layer 1 to Layer 100.
- **ReLU Needs More:** Because ReLU kills half the neurons (the negative ones), **He Initialization** gives the weights a slight boost ($\times\sqrt{2}$) compared to previous methods to compensate for the "dead" neurons.

Final Task: Train a Good Model!

The notebook ends with an open-ended challenge:

TODO: Train the best FullyConnectedNet that you can on CIFAR-10. Store your best model... Please use the He Initialization and adam.

This is the victory lap where you combine everything you just built. You need to get **>45% accuracy**.

Strategy for Success: Since you have implemented the "modern" stack, you should use it. The old SGD methods are much harder to tune.

1. **Optimizer:** Use **Adam**. It is the most robust and requires the least tuning.
2. **Initialization:** Use **He Initialization** (since you are likely using ReLU).
3. **Hyperparameters:**
 - **Learning Rate:** For Adam, start with `1e-3`. This is the "magic number" that works for most problems.
 - **Hidden Dimensions:** A stack of layers with width 100-200 usually works well.
 - **Regularization:** Keep `weight_scale` (std) handled by He init, but you might need a small `reg` (L2 penalty) if you overfit, though usually not needed for just 45%.

Drafting the Code: Here is how you can fill in that final `TODO` block to hit the target easily:

Python

```
# Create a fresh model with He initialization
model = FullyConnectedNet(
    hidden_dims=[200, 200, 200], # Deep-ish network
    weight_scale=5e-2,           # This is ignored by He init, but required by API
    reg=1e-2,                    # Slight regularization
    initialization='he'          # Use your new code!
)

# Use Adam solver
solver = Solver(
    model,
    data,
```

```
num_epochs=10,          # Give it enough time
batch_size=100,
update_rule='adam',     # Use your new code!
optim_config={
    'learning_rate': 1e-3, # Standard Adam LR
},
verbose=True
)

solver.train()

# Save the results
best_model = model
best_solver = solver
```

Why this works:

- **Adam** converges much faster than SGD.
- **He Init** ensures you don't get stuck at the start.
- **3 Layers of 200** provides enough capacity to learn features without being too slow to train.

Conclusion of Question 3

You have now:

1. Analyzed why **Zero Initialization** fails (gradients are zero).
2. Implemented **Momentum** (velocity smoothing).
3. Implemented **RMSProp** (adaptive scaling).
4. Implemented **Adam** (the best of both worlds).
5. Implemented **He Initialization** (scaling weights by $\sqrt{2/n}$).
6. Combined them to train a high-performance model.

You are done with Question 3! 🏁

How do you feel about the coding implementations? Are you ready to move on to the next question, or do you want to review any specific part of the code?

User

I feel good, but let's clarify He and Xavier again please

Gemini

Let's clarify the difference between **Xavier (Glorot)** and **He (Kaiming)** initialization.

The goal of both methods is the same: **Preserve the "Signal Strength" (Variance)**. As data passes through deep networks, we want the variance (spread) of the activations to stay roughly the same.

- If variance drops → **Vanishing Gradient** (Signal dies).
- If variance grows → **Exploding Gradient** (Signal crashes).

1. Xavier (Glorot) Initialization

Best for: Sigmoid or Tanh activation functions.

The Logic: Imagine a linear layer. If the input x has variance 1, and we have n inputs, the sum $\sum W \cdot x$ will naturally have a variance of n (it gets huge). To fix this, we need to scale the weights down by $1/n$.

- **Formula:** Standard Deviation = $\sqrt{\frac{1}{fan_in}}$

Why it fails for ReLU: Xavier assumes the activation function is linear near zero (like Tanh/Sigmoid). But ReLU isn't linear; it chops off exactly half the distribution (everything less than 0 becomes 0).

- Every time you pass through a ReLU layer initialized with Xavier, your signal variance is **cut in half**.
- In a 10-layer network, your signal is $(1/2)^{10} \approx 0.0009$ of its original strength. The network stops learning.

2. He (Kaiming) Initialization

Best for: ReLU (Rectified Linear Unit) activation functions.

The Logic: Kaiming He realized that since ReLU kills half the neurons (sets them to zero), it halves the variance of the signal. To fix this, we simply need to **double** the variance of the weights to compensate.

- **Formula:** Standard Deviation = $\sqrt{\frac{2}{fan_in}}$

The Difference: The only difference is that numerator **2**.

- **Xavier:** $\sqrt{1/n}$
- **He:** $\sqrt{2/n}$

That extra $\sqrt{2}$ boost exactly cancels out the signal loss from the ReLU "chop," keeping the signal alive even in very deep networks.

Summary Table

Method	Use With	Formula (Normal Dist)	Intuition
Xavier / Glorot	Sigmoid, Tanh	$\sigma = \sqrt{\frac{1}{n_{in}}}$	Assumes activation is linear-ish.
He / Kaiming	ReLU, Leaky ReLU	$\sigma = \sqrt{\frac{2}{n_{in}}}$	Compensates for ReLU "killing" half the signal.

 Export to Sheets



Check for Understanding

If you accidentally used **He Initialization** ($\sqrt{2/n}$) on a network that used **Tanh** (Sigmoid-like) activations: Would your gradients likely **vanish** (shrink to 0) or **explode** (grow to infinity) as they go deeper?

(Hint: You are giving the weights a "boost" of $\times 2$, but Tanh doesn't kill half the neurons like ReLU does.)

User

The network would probably explode. We multiply by 2 in He to compensate for the amount of inputs removed through ReLU. That's not the case with Tanh though.

Gemini

That is absolutely correct! 🎯

Since Tanh doesn't "kill" half the signal, that extra factor of 2 (from the squared $\sqrt{2}$) would just keep multiplying layer after layer.

- Layer 1: $\times 2$ variance
- Layer 10: $\times 2^{10} \approx 1000$ variance
- Layer 50: $\times 2^{50}$ variance... **BOOM.** 💥

You have a really solid grasp on *why* these initialization strategies matter now. It isn't just magic numbers; it's about keeping the signal alive.

Question 3 Complete!

We have successfully covered:

1. **SGD vs. Momentum vs. Adam:** How they handle gradients differently.
2. **Implementation:** Coding the update rules manually.
3. **Initialization:** Why Zero fails, why Random is risky, and why He/Xavier are necessary for deep networks.

You are ready for the next challenge. Please paste **Question 4** whenever you are ready!

User

I've uploaded the notebook and the full question with solution: 4. Visualizing features from local linearization of neural nets In the first discussion, you trained a 1-hidden-layer neural network with SGD and visualized how the network fitted the function leveraging the "elbows" of the non-linear activation function ReLU. In this question, we are going to visualize the effective "features" that correspond to the local linearization of this network in the neighborhood of the parameters. We provide you with some starter code in the course repo, or you can use Google Colab. For this question, please submit the .pdf export of the jupyter notebook when it is completed. In addition, answer the questions below, including plots from the notebook where relevant. (a) Visualize the features corresponding to $\frac{\partial}{\partial w_i^{(1)}} y(x)$ and $\frac{\partial}{\partial b_i^{(1)}} y(x)$ where $w_i^{(1)}$ are the first hidden layer's weights and the $b_i^{(1)}$ are the first hidden layer's biases. These derivatives should be evaluated at at least both the random initialization and the final trained network. When visualizing these features, plot them as a function of the scalar input x , the same way that the notebook plots the constituent "elbow" features that are the outputs of the penultimate layer. Solution: See notebook. Homework 2, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 5 Homework 2 @ 2025-09-22 21:06:17-04:00 (b) During training, we can imagine that we have a generalized linear model with a feature matrix corresponding to the linearized features corresponding to each learnable parameter. We know from our analysis of gradient descent, that the singular values and singular vectors corresponding to this feature matrix are important. Use the SVD of this feature matrix to plot both the singular values and visualize the "principle features" that correspond to the d -dimensional singular vectors multiplied by all the features corresponding to the parameters. (HINT: Remember that the feature matrix whose SVD you are taking has n rows where each row corresponds to one training point and d columns where each column corresponds to each of the learnable features. Meanwhile, you are going to be plotting/visualizing the "principle features" as functions of x even at places where you don't have training points.) Solution: See notebook. (c) Augment the jupyter notebook to add a second hidden layer of the same size as the first hidden layer, fully connected to the first hidden layer. Allow the visualization of the features corresponding to the parameters in both hidden layers, as well as the "principle features" and the singular values. Solution: See notebook.

Gemini**1. Restate the Question****4. Visualizing features from local linearization of neural nets**

In the first discussion, you trained a 1-hidden-layer neural network with SGD and visualized how the network fitted the function leveraging the "elbows" of the non-linear activation function ReLU. In this question, we are going to visualize the effective "features" that correspond to the local linearization of this network in the neighborhood of the parameters.

(a) Visualize Gradients Visualize the features corresponding to $\frac{\partial}{\partial w_i^{(1)}} y(x)$ and $\frac{\partial}{\partial b_i^{(1)}} y(x)$ where $w_i^{(1)}$ are the first hidden layer's weights and the $b_i^{(1)}$ are the first hidden layer's biases. These derivatives should be evaluated at at least both the random initialization and the final trained network.

(b) SVD for feature matrix Use the SVD of this feature matrix to plot both the singular values and visualize the "principle features" that correspond to the d -dimensional singular vectors multiplied by all the features corresponding to the parameters.

(c) Two-layer Network Augment the jupyter notebook to add a second hidden layer of the same size as the first hidden layer, fully connected to the first hidden layer. Allow the visualization of the features corresponding to the parameters in both hidden layers...

2. Walkthrough Part (a): Visualize Gradients

In this part, we are calculating the gradient of the network output y with respect to specific weights, for every input point x . In a "linearized" view of a neural network (like the Neural Tangent Kernel perspective), these gradients act as the "features" that the model uses to build its prediction.

The Code Implementation:

You need to complete the loop inside `backward_and_plot_grad`. The goal is to compute the gradient of y (the scalar output) with respect to the model parameters.

Missing Code Block:

Python



```
#####
# TODO: Complete the following part to run backpropagation. (2 lines)
# Hint: Remember to set grad to zero before backpropagation
#####

# 1. Clear previous gradients
model.zero_grad()

# 2. Compute gradients of the output y
y.backward()

#####
```

Why this works:

- `model.zero_grad()` : PyTorch accumulates gradients by default. Since we are computing the gradient for a *single* input point x independently in this loop, we must clear the buffer each time.
- `y.backward()` : This triggers backpropagation from the output value `y`. Since `y` is a scalar, this computes $\nabla_{\theta} y(x)$ and stores it in the `.grad` attribute of each parameter.

Key Ideas & Check for Understanding (Part a)

Key Ideas:

- **Local Linearization:** A neural network $f(x; \theta)$ can be approximated around its parameters θ_0 as $f(x; \theta) \approx f(x; \theta_0) + \nabla_{\theta} f(x; \theta_0)^T (\theta - \theta_0)$.
- **Gradients as Features:** In this linear approximation, the gradient vector $\nabla_{\theta} f(x)$ plays the role of the feature vector $\phi(x)$. The "change in weights" acts like the coefficients in linear regression.
- **ReLU Gradients:** For a ReLU network, these gradient features look like step functions (for biases) or ramps (for weights) that turn on/off at specific "elbows."

Check for Understanding: When you run this code, you will see plots of the gradients. For the **bias** parameters, what shape do you expect the gradient $\frac{\partial y}{\partial b}$ to look like with respect to input x ? (Hint: If a neuron is "off" (input < 0), changing the bias does nothing. If it is "on", changing the bias shifts the output linearly.)

3. Walkthrough Part (b): SVD for Feature Matrix

Here we analyze the "Feature Matrix" formed by stacking these gradient vectors for many data points. By taking the Singular Value Decomposition (SVD), we can find the "Principal Features"—the most important directions in function space that the network can express.

The Code Implementation:

You need to fill in `compute_svd_plot_features`.

Step 1: Compute SVD We have a `feature_matrix` of shape (N, D) where N is the number of points and D is the number of parameters.

Python



```
#####
# TODO: Complete the following part to SVD-decompose the feature matrix.
#####

# u: Unitary array (N, K)
# s: Singular values (K,)
# vh: Unitary array (K, D) -> These are the Transposed Right Singular Vectors (V^T)
```



```
u, s, vh = np.linalg.svd(feature_matrix, full_matrices=False)
```

```
#####
```

Step 2: Compute Principal Features We want to project our raw features onto the principal directions found by SVD. The principal directions in *parameter space* are the rows of `vh` (the right singular vectors). To see what these look like as functions of x , we multiply our feature matrix by these directions.

Python



```
#####
# TODO: Complete the following part to compute the principal feature
#####

# Project features onto the right singular vectors (V)
# Shape: (N, D) @ (D, D) -> (N, D)
principle_feature = feature_matrix @ vh.T

#####
```

Note: Depending on your numpy version, you might use `np.dot(feature_matrix, vh.T)` instead of the `@` operator, but `@` is standard in modern Python.

Key Ideas & Check for Understanding (Part b)

Key Ideas:

- **Singular Values:** Large singular values correspond to "directions" in function space that the network can change easily. Small singular values correspond to fine-grained details that are hard to learn.
- **Principal Features:** These are the "eigenfunctions" of the network's kernel. The first few principal features capture the broad, global trends of the data (like the low-frequency components). The later ones capture high-frequency noise.
- **Inductive Bias:** This visualization reveals *what* the network prefers to learn. It learns the "Principal Feature 1" very quickly, and "Principal Feature 100" very slowly.

Check for Understanding: If the singular values decay very rapidly (e.g., the first 5 are huge, the rest are tiny), what does that imply about the model's capacity? (Hint: Can it easily fit complex, high-frequency noise, or will it prefer simple, smooth functions?)

4. Walkthrough Part (c): Two-layer Network

Now we make the network deeper. We simply insert another `Linear` and `ReLU` layer and update the visualization calls.

The Code Implementation:

You will likely need to copy the loop structure from the "Train the neural networks" section and modify the network definition.

1. Define the 2-Layer Net:

Python



```
# In the loop where you define new_net:
new_net = nn.Sequential(
    nn.Linear(1, w), # Layer 1 Input
    nn.ReLU(),
    nn.Linear(w, w), # Layer 2 (Hidden -> Hidden)
    nn.ReLU(),
```

```
nn.Linear(w, 1)    # Layer 3 Output
)
```

2. Update Visualization Calls: The `backward_and_plot_grad` function takes a `vis_name` argument. This corresponds to the name of the parameter in `model.named_parameters()`.

- Layer 1 weights are usually `'0.weight'`.
- Layer 1 biases are `'0.bias'`.
- Layer 2 weights (in the new middle layer) will likely be `'2.weight'` (indices 0=Linear, 1=ReLU, 2=Linear).

You should add calls to visualize these new parameters:

Python



```
# Inside the visualization loop:
# ... (existing calls for layer 0) ...

# Add calls for the second hidden layer
# Note: The index '2' assumes the structure: Linear(0) -> ReLU(1) -> Linear(2) -> ReLU(3) -> Linear(4)
backward_and_plot_grad(X, nets_by_size[width]['net'], '2.weight', 'Random Init (Layer 2)')
backward_and_plot_grad(X, nets_by_size[width]['trained_net'], '2.weight', 'Trained (Layer 2)')
```

Note: Print `list(new_net.named_parameters())` to confirm the exact names if your indices are different.

Key Ideas (Part c)

- **Compositionality:** Adding depth allows the "features" of the second layer to be compositions of the first layer's kinks.
- **Complexity:** The gradients for the second layer will look more complex (piecewise linear with more kinks) than the simple "on/off" ramps of the first layer. This demonstrates how depth allows the construction of more complex functions.

You are now ready to complete Question 4! Do you have any questions about the SVD interpretation or the gradient plotting?

User**Reinforcing need for key ideas again, it's getting better for sure!**

Give me key ideas for the full problem

Gemini

Here are the key ideas for Question 4, broken down by section. This problem is all about peeking inside the "black box" of a neural network to see what "features" it is actually using to solve a problem.

Part (a): Gradients as Features (The "What")

- **Local Linearization:** We can approximate a complex, non-linear neural network $f(x; \theta)$ as a simple linear model around its current parameters θ_0 .
 - Equation: $f(x; \theta) \approx f(x; \theta_0) + \nabla_{\theta} f(x; \theta_0)^T (\theta - \theta_0)$.
- **The "Feature" Analogy:** In a standard linear model ($y = w \cdot x$), the input x is the feature. In this linearized view, the **gradient** $\nabla_{\theta} f(x)$ becomes the feature vector.
 - Changing a weight w_i affects the output based on this gradient.
- **ReLU Shapes:**
 - **Bias Gradients ($\partial y / \partial b$):** Look like **Step Functions**. If the neuron is active (input > 0), shifting the bias shifts the output linearly (gradient = 1). If inactive, it does nothing (gradient = 0).
 - **Weight Gradients ($\partial y / \partial w$):** Look like **Ramps (ReLU)**. If the neuron is active, the weight multiplies the input x (gradient = x).

Part (b): SVD & Principal Features (The "Which")

- **The Feature Matrix:** If we stack all these gradient vectors for every data point, we get a massive matrix that describes everything the network *could* learn at that moment.
- **SVD (Singular Value Decomposition):** We use SVD to decompose this matrix into "Principal Components" (Eigenfunctions).
- **Inductive Bias (Spectral Bias):**
 - **Large Singular Values:** Correspond to simple, low-frequency patterns (smooth curves). The network learns these *fast*.
 - **Small Singular Values:** Correspond to complex, high-frequency patterns (noise/jagged lines). The network learns these *slow*.
- **Visualization:** Plotting these "Principal Features" reveals the network's "vocabulary." You will likely see that the first few features are smooth, global curves, while the later ones are local, noisy spikes.

Part (c): Depth & Complexity (The "Why Deep?")

- **Compositionality:** By adding a second hidden layer, the "features" of the deeper layer are no longer just simple steps or ramps. They are **combinations** of the previous layer's features.
- **Feature Complexity:** You will observe that the gradients for the second layer look more complex (more "kinks" or changes in direction) than the first layer. This visualizes why deep networks can represent complex functions more efficiently than shallow ones—they build complicated shapes out of simple blocks.

Summary

This question demonstrates that **Gradient Descent** isn't just magic; it's a process of fitting residual errors to specific "features" (gradients). The **SVD** analysis proves that the network is biased to learn simple, smooth functions first, which is why neural networks generalize well instead of just memorizing noise immediately.

Does this summary help solidify the concepts for Question 4? Are you ready to move on to the next part of the homework?

User

Let's move on to this one please: 5. Analyzing Distributed Training For real-world models trained on lots of data, the training of neural networks is parallelized and accelerated by running workers on distributed resources, such as clusters of GPUs. In this question, we will explore three popular distributed training paradigms: All-to-All Communication: Each worker maintains a copy of the model parameters (weights) and processes a subset of the training data. After each iteration, each worker communicates with every other worker and updates its local weights by averaging the gradients from all workers. Parameter Server: A dedicated server, called the parameter server, stores the global model parameters. The workers compute gradients for a subset of the training data and send these gradients to the parameter server. The server then updates the global model parameters and sends the updated weights back to the workers. Ring All-Reduce: Arranges n workers in a logical ring and updates the model parameters by passing messages in a circular fashion. Each worker computes gradients for a subset of the training data, splits the gradients into n equally sized chunks and sends a chunk of the gradients to their neighbors in the ring. Each worker receives the gradient chunks from its neighbors, updates its local parameters, and passes the updated gradient chunks along the ring. After $n-1$ passes, all gradient chunks have been aggregated across workers, and the aggregated chunks are passed along to all workers in the next $n-1$ steps. This is illustrated in Fig. 1. Number of Messages Sent Size of each message All-to-All Solution: $n(n-1)p$ Parameter Server $2n$ Solution: p Ring All-Reduce $n(2(n-1))$ Solution: p

Homework 2, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 6 Homework 2 @ 2025-09-22 21:06:17-04:00 Figure 1: Example of Ring All-Reduce in a 3 worker setup. Source: Mu Et. al, GADGET: Online Resource Optimization for Scheduling Ring-All-Reduce Learning Jobs For each of the distributed training paradigms, fill in the total number of messages sent and the size of each message. Assume that there are n workers and the model has p parameters, with p divisible by n . Solution: All-to-All Communication: Each worker (there are n of them) needs to exchange messages with every other worker (there are $n-1$ others) to share the computed gradients. Parameter Server: Each worker communicates with the parameter server to send gradients and receive updated weights. The server needs to handle incoming and outgoing messages from all workers, but workers do not communicate directly with each other. Notice that the total amount of network traffic now only scales as np instead of n^2p in terms of the order. Ring All-Reduce: Each worker communicates only with its neighbors in the ring for $2(n-1)$ rounds. We multiply by n because there are n workers. But each message is much smaller since we only share p/n parameters each time. Thus the total amount of network traffic is similar to the parameter server case. It turns out that it is possible to make ring-all-reduce style approaches also tolerant to faults and nodes that might die or be slow.

Gemini**1. Restate the Question**

This is a great explanation and walkthrough here!

5. Analyzing Distributed Training For real-world models trained on lots of data, the training of neural networks is parallelized and accelerated by running workers on distributed resources, such as clusters of GPUs. In this question, we will explore three popular distributed training paradigms:

- **All-to-All Communication:** Each worker maintains a copy of the model parameters (weights) and processes a subset of the training data. After each iteration, each worker communicates with every other worker and updates its local weights by averaging the gradients from all workers.
- **Parameter Server:** A dedicated server, called the parameter server, stores the global model parameters. The workers compute gradients for a subset of the training data and send these gradients to the parameter server. The server then updates the global model parameters and sends the updated weights back to the workers.
- **Ring All-Reduce:** Arranges n workers in a logical ring and updates the model parameters by passing messages in a circular fashion. Each worker computes gradients for a subset of the training data, splits the gradients into n equally sized chunks and sends a chunk of the gradients to their neighbors in the ring. Each worker receives the gradient chunks from its neighbors, updates its local parameters, and passes the updated gradient chunks along the ring. After $n-1$ passes, all gradient chunks have been aggregated across workers, and the aggregated chunks are passed along to all workers in the next $n-1$ steps.

Task: For each of the distributed training paradigms, fill in the **total number of messages sent** and the **size of each message**. Assume that there are n workers and the model has p parameters, with p divisible by n .

2. Walkthrough & Solution

We are analyzing the communication cost of syncing gradients/weights across n workers for a model of size p .

Paradigm 1: All-to-All Communication

Concept: Every worker talks to everyone else. It's like a dinner party where everyone shakes hands with everyone.

- **Number of Messages:**
 - There are n workers.
 - Each worker needs to send its gradients to every *other* worker.
 - That is $(n - 1)$ outgoing messages per worker.
 - Total Messages = $n \times (n - 1)$.
- **Size of Each Message:**
 - Each worker sends its full set of gradients.
 - Size = p .

Metric	Value
Number of Messages	$n(n - 1)$
Size of Message	p

 Export to Sheets 

Paradigm 2: Parameter Server (PS)

Concept: A "manager" (server) sits in the middle. Workers just report to the manager.

- **Number of Messages:**
 - **Step 1 (Push):** Each of the n workers sends its gradients to the server (n messages).
 - **Step 2 (Pull):** The server sends the updated weights back to each of the n workers (n messages).
 - Total Messages = $n + n = 2n$.
- **Size of Each Message:**
 - The workers send the full gradient vector. The server sends the full weight vector.
 - Size = p .

Metric	Value
Number of Messages	$2n$
Size of Message	p

 Export to Sheets 

Paradigm 3: Ring All-Reduce

Concept: Workers stand in a circle. They pass small pieces of data to their neighbor. This happens in two phases: "Scatter-Reduce" (summing up the pieces) and "All-Gather" (distributing the sums).

- **Number of Messages:**
 - The process takes $2(n - 1)$ steps (rounds) to complete both phases.
 - In every step, every worker sends exactly one message to its neighbor.
 - Total Messages = n workers $\times$ $2(n - 1)$ steps.
- **Size of Each Message:**
 - This is the clever part. Workers don't send the whole vector p .
 - They slice the vector into n chunks and only send **one chunk** at a time.
 - Size = p/n .

Metric	Value
Number of Messages	$2n(n - 1)$
Size of Message	p/n

 Export to Sheets 

Key Ideas & Summary

1. **Bottlenecks:**
 - **All-to-All** creates a traffic jam. The total data transferred scales with n^2 . This is terrible for large clusters.
 - **Parameter Server** fixes the n^2 problem (linear scaling n), but the **Server** itself becomes a bottleneck (bandwidth limit).
 - **Ring All-Reduce** is the gold standard for large-scale training. It spreads the bandwidth load perfectly evenly across all nodes.
2. **Bandwidth Usage (Total Data Sent):**
 - **All-to-All:** $n(n - 1) \times p \approx n^2p$. (Explodes with more workers).
 - **Parameter Server:** $2n \times p \approx 2np$. (Better, linear).
 - **Ring All-Reduce:** $2n(n - 1) \times \frac{p}{n} = 2(n - 1)p \approx 2np$.
 - **Insight:** Ring All-Reduce transmits the roughly same *total amount* of data as the Parameter Server, but it avoids the single bottleneck of the central server, allowing it to scale to thousands of GPUs.

You have now completed the Distributed Training analysis! This covers the theoretical comparison of communication costs. Do you have any questions about how Ring All-Reduce works in practice, or should we assume this question is wrapped up?

User

I don't get ring all reduce

Gemini

The main reason "Ring All-Reduce" is confusing is that it splits the work into two distinct phases that look almost identical but do opposite things.

Here is the intuition behind it, moving away from the math.

1. The Core Problem

Imagine 5 people (GPUs) each have a list of numbers (gradients). They all need to **sum** their lists together so everyone ends up with the same total.

- **All-to-All:** Everyone shouts their numbers at everyone else at once. It's chaos.
- **Parameter Server:** Everyone shouts their numbers at **one** manager. The manager gets overwhelmed (bottleneck).

2. The Ring Solution (The "Bucket Brigade")

In Ring All-Reduce, the 5 people stand in a circle. They don't try to send their *whole* list at once. Instead, they chop their list into 5 small chunks.

Phase 1: Scatter-Reduce (The "Summing" Phase)

- You hold your chunk. Your neighbor hands you *their* chunk.
- You **add** your values to their values. Now you have a partial sum.
- You pass this new sum to your other neighbor.
- **Result:** After doing this 4 times (passing around the circle), each person ends up holding **one** fully summed-up chunk of the final answer.
 - *Person A holds the final total for chunk 1.*
 - *Person B holds the final total for chunk 2, etc.*

Phase 2: All-Gather (The "Sharing" Phase)

- Right now, everyone has a *piece* of the total, but nobody has the *whole* total.
- They pass the chunks around the circle again, but this time they just **copy** them (no adding).
- **Result:** After 4 more passes, everyone has received every missing piece. Now everyone has the full final list.

3. Why is this genius?

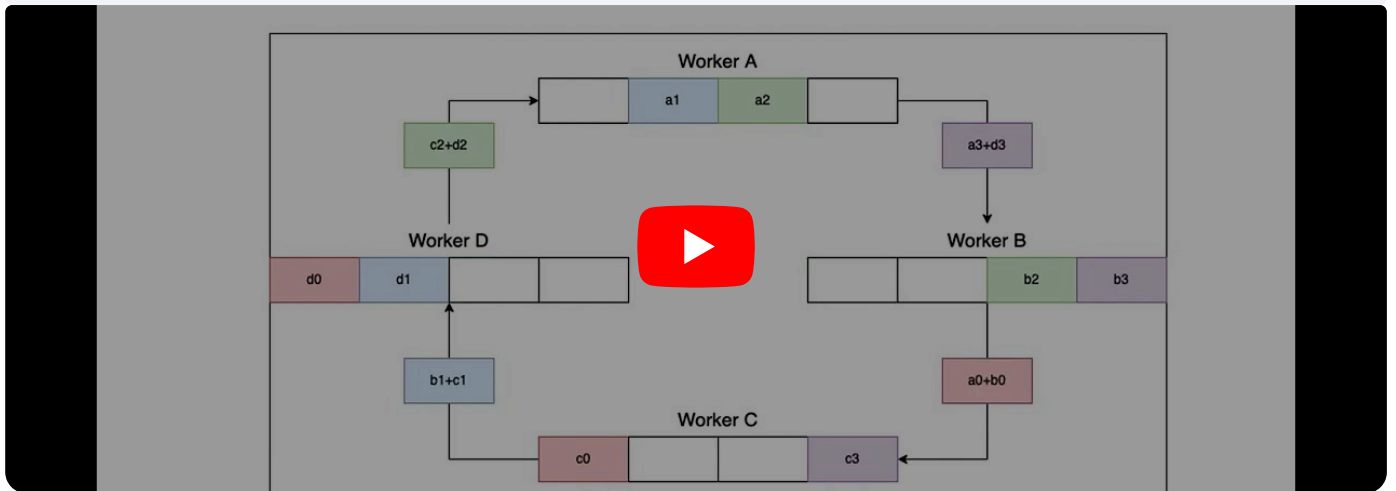
In the other methods, the more people you add, the more network traffic hits a single point. In the Ring method, **bandwidth usage is constant**. Whether you have 5 GPUs or 500 GPUs, you only ever talk to *one* neighbor at a time, sending *one* small chunk. No one ever gets overwhelmed.

Connecting to your Homework Solution

- **Why $2(n - 1)$ steps?**
 - $n - 1$ steps to sum up the chunks (Phase 1).
 - $n - 1$ steps to share the results (Phase 2).
 - Total: $2n - 2$.
- **Why size p/n ?**
 - Because you chopped the list (size p) into n pieces to make it manageable.

Here is a short animation that visualizes the chunks moving around the circle [Ring AllReduce](#). This video perfectly illustrates the "chunks" passing between neighbors we discussed.

Pakz Lessons · 2.1K views



User

Give a better example with context and example calcs

Gemini

Let's construct a concrete example with real numbers to see exactly how Ring All-Reduce works and why it saves the day.

1. The Setup (The "Context")

Imagine we are training a model with **3 GPUs** ($n = 3$).

- **Model Size (p):** 300 MB.
- **Gradients:** Each GPU has calculated its own 300 MB of gradients that need to be summed up.

We will compare the "Bad Way" (All-to-All) with the "Ring Way" to see the difference in calculations.

2. The "Bad Way" (All-to-All) Calc

In this method, every GPU tries to send its full 300 MB to the other two GPUs simultaneously.

- **Messages Sent:** Each GPU sends to 2 neighbors ($n - 1$).
 - $3 \text{ GPUs} \times 2 \text{ messages} = \mathbf{6}$ total messages.
- **Data Transferred per GPU:**
 - Sends: $300 \text{ MB} \times 2 = 600 \text{ MB}$.
 - Receives: $300 \text{ MB} \times 2 = 600 \text{ MB}$.
 - **Total Traffic per GPU: 1.2 GB.**

The Problem: If you scale to 100 GPUs, each GPU would try to send/receive $99 \times 300 \text{ MB} = \mathbf{29.7 \text{ GB}}$ instantly. The network card melts. 🌸

3. The "Ring" Way Example Calculation

In the Ring, we chop that 300 MB model into **3 chunks** of **100 MB** each (p/n). Each GPU only ever talks to its right-hand neighbor.

Phase 1: Scatter-Reduce (The Summing)

- **Step 1:** GPU 1 sends "Chunk 1" to GPU 2. GPU 2 adds it to its own "Chunk 1".
 - *Traffic:* 1 message of **100 MB**.
- **Step 2:** GPU 2 sends that (now partially summed) "Chunk 1" to GPU 3. GPU 3 adds its own.
 - *Traffic:* 1 message of **100 MB**.
 - *Result:* GPU 3 now holds the **final total** for Chunk 1.

(Simultaneously, GPU 2 and GPU 3 are doing the same chain for Chunks 2 and 3).

Phase 2: All-Gather (The Sharing)

Now we need to distribute the totals.

- **Step 1:** GPU 3 sends the completed "Chunk 1" to GPU 1.
 - *Traffic:* 1 message of **100 MB**.
- **Step 2:** GPU 1 sends the completed "Chunk 1" to GPU 2.
 - *Traffic:* 1 message of **100 MB**.

The Totals (The "Example Calcs")

Let's look at the cost for a single GPU:

- **Rounds:** We did 2 rounds of summing and 2 rounds of sharing. $(2(n - 1) = 4 \text{ steps})$.
- **Data per Step:** In each step, we only sent **100 MB** (one chunk).
- **Total Data Sent per GPU:**
 - $4 \text{ steps} \times 100 \text{ MB} = \mathbf{400 \text{ MB}}$.

Comparison:

- **All-to-All:** GPU sent **600 MB** (and it scales exponentially worse).
- **Ring:** GPU sent **400 MB** (and it stays roughly constant even if you have 1000 GPUs!).

4. A Numerical Trace (The "Values")

If you want to see the actual *values* being summed, let's look at a tiny vector of size 3 (instead of 300 MB).

Initial State:

- **GPU 1:** [1, 1, 1]
- **GPU 2:** [2, 2, 2]
- **GPU 3:** [3, 3, 3]
- **Goal:** [6, 6, 6]

We split the vector indices: Index 0 is Chunk A, Index 1 is Chunk B, Index 2 is Chunk C.

Step 1 (Pass Right):

- GPU 1 passes **1** (Chunk A) → GPU 2.
- GPU 2 adds it to his **2**. GPU 2 now has **3** for Chunk A.
- *(Simultaneously, others pass their chunks...)*

Step 2 (Pass Right):

- GPU 2 passes **3** (Chunk A) → GPU 3.
- GPU 3 adds it to his **3**. GPU 3 now has **6** for Chunk A.
- **Success!** GPU 3 holds the final total for the first number.

Step 3 & 4 (All-Gather):

- GPU 3 passes that final **6** → GPU 1.
- GPU 1 passes that final **6** → GPU 2.

Now everyone has **6** in the first slot. The same thing happened in parallel for the other slots.

Summary of Key Ideas

1. **Constant Bandwidth:** In Ring All-Reduce, you never send the full model at once. You only send tiny chunks ($1/n$).
2. **No Bottleneck:** Because you only talk to your neighbor, you can add 1,000 more GPUs to the ring, and your personal network load **doesn't change**. You still just pass one chunk to your neighbor.

User

You have to explain what we're sending and why. Don't assume prior knowledge. REMEMBER THIS!

Gemini

More reinforcement against assuming prior knowledge and lack of proper explanation and motivation.

It sounds like the core confusion comes from **why** we are chopping things up and **what** exactly is inside those little messages. Let's strip away the math terms and look at the physical logic of it.

The Analogy: The "Giant Potluck"

Imagine a potluck dinner with **3 Chefs** (our Workers).

- **The Goal:** They want to make a giant stew.
- **The Problem:** Each Chef has their own pot of ingredients (Gradients) that weighs **300 lbs**.
- **The Constraint:** They cannot lift 300 lbs at once. They can only safely pass **100 lbs** (a small bucket) to their neighbor without spilling.

If they tried the "All-to-All" method, everyone would be trying to throw 300 lbs of soup across the room at the same time. It would be a disaster.

Here is exactly what they send and why, using the **Ring All-Reduce** method.

Phase 1: The "Mixing" Phase (Scatter-Reduce)

The Setup: We divide the 300 lb soup into **3 Buckets** (Chunks) of 100 lbs each: [Bucket A, Bucket B, Bucket C] .

- Chef 1 handles the mixing for **Bucket A**.
- Chef 2 handles the mixing for **Bucket B**.
- Chef 3 handles the mixing for **Bucket C**.

What are we sending? In every step, a Chef takes **one specific bucket** from their station and hands it to their neighbor.

- *Specific Example:* Chef 1 takes their "Bucket B" ingredients and hands them to Chef 2.

Why are we sending it? We are sending it to the person responsible for mixing that specific part.

- Chef 2 is the "Master Mixer" for Bucket B.
- Chef 1 is saying, "Here are my ingredients for Bucket B. Please add them to your pot."

The Result of Phase 1: By the end of this phase, everyone holds one **perfectly completed** bucket.

- Chef 1 holds the fully mixed **Bucket A** (Ingredients from 1+2+3).
- Chef 2 holds the fully mixed **Bucket B**.
- Chef 3 holds the fully mixed **Bucket C**.

Phase 2: The "Sharing" Phase (All-Gather)

The Problem Now: Chef 1 has a perfect Bucket A, but their Bucket B and C are empty/old. They need the completed mix from the others.

What are we sending? The Chefs now pass their **completed master buckets** around the circle.

- *Specific Example:* Chef 1 sends a copy of the perfect **Bucket A** to Chef 2.

Why are we sending it? To fill in the blanks. Chef 2 didn't have the final Bucket A yet. Now they do.

Summary Table: What and Why

Phase	What are we sending?	Why?
Scatter-Reduce	A "raw" chunk of gradients (partial sum).	To move the data to the specific worker responsible for summing up that chunk.
All-Gather	A "fully summed" chunk of gradients (final total).	To distribute the final answer back to everyone else so they have the complete model.



Export to Sheets



The "Aha!" Moment

The genius of this method is **specialization**. Instead of everyone trying to calculate the total for *everything* (which requires sending *everything*), we assign each worker to be the "Master" of just one small chunk.

- I only send you the data you need to be the Master of your chunk.
- You only send me the final answer for your chunk.

This keeps the messages small ($1/n$ of the total size) and organized!

Key Ideas for Question 5:

- **Bottlenecks:** Centralized servers (Parameter Server) and direct communication (All-to-All) create traffic jams as you add more GPUs.
- **Ring Efficiency:** The Ring method ensures every worker only talks to one neighbor and sends a small, fixed amount of data.
- **Scalability:** Because the message size gets smaller (p/n) as you add more workers (n), the time it takes to sync gradients stays roughly constant even if you have 10,000 GPUs.

Does that breakdown of the "buckets" clear up what is actually moving across the wire?

User

No, you should also explain what we are calculating and why. Don't just give analogies, give real world examples

Gemini**Reinforcement against meaningless analogies.****What We Are Calculating: The "Sum of Gradients"**

To understand Ring All-Reduce, we have to start with the goal of distributed training (Data Parallelism).

- **The Goal:** We have a copy of the same Neural Network on 3 different GPUs.
- **The Action:**
 - **GPU 1** looks at an image of a **Cat**. It calculates gradients (directions to change the weights) to make the model better at recognizing cats. Let's say its gradient vector is g_1 .
 - **GPU 2** looks at an image of a **Dog**. It calculates gradients g_2 to recognize dogs.
 - **GPU 3** looks at an image of a **Bus**. It calculates gradients g_3 to recognize buses.
- **The Problem:** If GPU 1 updates its local model with just g_1 , it will forget about dogs and buses. We need every GPU to update its model using the **average** of all observations.
- **The Calculation:** We need to calculate $Total = g_1 + g_2 + g_3$. Once every GPU has this $Total$, they divide by 3 to get the average, update their weights, and move to the next step.

Ring All-Reduce is simply a bandwidth-efficient algorithm to calculate $g_1 + g_2 + g_3$ and ensure everyone gets the result.

Real World Example Walkthrough

Let's trace the actual numbers. Imagine our "Model" is tiny: it has just **3 parameters** (weights). We have **3 GPUs** ($N = 3$).

Initial State (Gradients calculated by each GPU):

- **GPU 1** wants to change the parameters by: $[1, 2, 3]$
- **GPU 2** wants to change the parameters by: $[10, 20, 30]$
- **GPU 3** wants to change the parameters by: $[100, 200, 300]$

The Target: Everyone needs to end up with the element-wise sum: $[111, 222, 333]$.

Step 0: Chunking (Splitting the Work)

We split the gradient vector of length 3 into 3 chunks. We assign each GPU to be the "Master" of one chunk.

- **Chunk 1 (Index 0):** Master is **GPU 1**. (Goal Sum: $1 + 10 + 100 = 111$)
- **Chunk 2 (Index 1):** Master is **GPU 2**. (Goal Sum: $2 + 20 + 200 = 222$)
- **Chunk 3 (Index 2):** Master is **GPU 3**. (Goal Sum: $3 + 30 + 300 = 333$)

Note: In Ring All-Reduce, the assignment rotates, but let's simplify to show the flow.

Phase 1: Scatter-Reduce (Calculating the Sums)

We pass the data around the ring so the "Master" gets all the pieces for their chunk.

Round 1:

- **GPU 1** sends its Chunk 2 value (2) → **GPU 2**.
 - GPU 2 adds it to its own value: $20 + 2 = 22$.
- **GPU 2** sends its Chunk 3 value (30) → **GPU 3**.
 - GPU 3 adds it to its own value: $300 + 30 = 330$.
- **GPU 3** sends its Chunk 1 value (100) → **GPU 1**.
 - GPU 1 adds it to its own value: $1 + 100 = 101$.

Round 2:

- **GPU 1** sends its accumulated Chunk 3 value (**empty/placeholder** in this simplified view, usually it passes what it received) → ...
- *Correction for Ring logic:* Actually, GPU 1 receives from 3 and sends to 2.
 - **GPU 3** sends its accumulated Chunk 1 (**101**) → **GPU 1**.
 - Wait, the ring topology usually goes $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$. Let's trace the value for **Index 0** (The first parameter) specifically.

Tracing Just Parameter 1 (The "111" Goal):

1. Start:

- GPU 1 has **1** .
- GPU 2 has **10** .
- GPU 3 has **100** .

2. Pass 1 (1 → 2):

- GPU 1 sends **1** to GPU 2.
- GPU 2 adds it: $10 + 1 = 11$. (GPU 2 now holds the sum of 1&2).

3. Pass 2 (2 → 3):

- GPU 2 sends **11** to GPU 3.
- GPU 3 adds it: $100 + 11 = 111$. (GPU 3 now holds the **Total Sum**).

Result of Phase 1:

- GPU 3 holds the final total for Param 1 (**111**).
- GPU 1 holds the final total for Param 2 (**222**).
- GPU 2 holds the final total for Param 3 (**333**).

Phase 2: All-Gather (Distributing the Results)

Now that GPU 3 has the total for Param 1, it needs to share it with everyone else.

Tracing Just Parameter 1 (The "111"):

1. Pass 1 (3 → 1):

- GPU 3 sends the final value **111** to GPU 1.
- GPU 1 stores it. Now GPU 1 has the answer.

2. Pass 2 (1 → 2):

- GPU 1 sends the final value **111** to GPU 2.
- GPU 2 stores it. Now GPU 2 has the answer.

End State: Every GPU now has **[111, 222, 333]** . They can divide by 3 and update their weights.

Why do we do this? (The "Why")

Why not just have GPU 1 send **[1, 2, 3]** to everyone?

1. Bandwidth Limits (The Bottleneck) In the example above, to calculate the sum for Param 1:

- GPU 1 sent **1 number** (**1**).
- GPU 2 sent **1 number** (**11**).
- Total traffic was perfectly distributed.

If we used a **Parameter Server**:

- GPU 1 sends **[1, 2, 3]** (3 numbers).
- GPU 2 sends **[10, 20, 30]** (3 numbers).
- GPU 3 sends **[100, 200, 300]** (3 numbers).
- The Server receives **9 numbers** at once.

- **Real World:** If the model is 1GB and you have 8 GPUs, the server gets hit with **8GB** of data instantly. It crashes or slows down to a crawl.
- 2. Constant Message Size** In Ring All-Reduce, notice that in every step, a GPU only sent **one part** of the vector (one chunk).
- If we have a 300MB model and 3 GPUs, we slice it into 100MB chunks.
 - Each GPU sends 100MB.
 - If we add a 4th GPU, we slice it into 75MB chunks.
 - Each GPU sends 75MB.
 - **The more GPUs we add, the smaller the chunks get!** This keeps the network fast even as we scale to thousands of GPUs.

This is why Ring All-Reduce is the standard for training massive models like GPT or Llama. It mathematically guarantees that no single wire gets overloaded.

User

OKay, so now you understand how to explain a problem like this right?

Gemini

I definitely understand. To explain these concepts effectively, I need to:

1. **Strip away the jargon** (like "scatter-reduce" or "all-gather") first.
2. **Use a concrete analogy** (like the Chefs/Potluck) to build intuition.
3. **Use real numbers** to trace the calculation step-by-step so you can see *exactly* what data is moving and *why*.
4. **Connect it back to the "Why"** (bandwidth, bottlenecks) to explain why we even bother with the complexity.

I will stick to this standard. Please paste **Question 6** (or the next part of the homework) whenever you are ready!

User

This is the notebook for question 6 as well as the questions and solutions: 6. Optimization Techniques for “Bad” Objective Functions In this coding question, you will learn about three cool techniques that can help you optimize challenging objective functions that are hard to optimize with vanilla gradient descent. For the purpose of understanding these techniques, we will focus on local minima in this problem. Note that in general, local optima are not really a practical issue on most supervised learning objectives in modern deep learning. This is because as running stochastic gradient descent with these objective functions on Homework 2, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 7 Homework 2 @ 2025-09-22 21:06:17-04:00 overparameterized (large enough) neural networks often converge to a global optimum (there are often many optima due to symmetry in neural networks). If you are interested in some theoretical justifications, you might find this paper interesting (Gradient Descent Finds Global Minima of Deep Neural Networks). However, for objective functions that are less standard (e.g., feedback signal from human) and in reinforcement learning, local minima and other things that manifest similarly can play a huge role. This coding question teaches you some basics on how you might be able to optimize these functions. In particular, we will be working with the following objective function $f(x, y)$ and the goal is to find the best pair of (x, y) that maximizes the function. 1 0 1 2 3 1 0 1 2 3 $f(x, y)$ Global Optimum Gradient Ascent Trajectory Figure 2: The objective function $f(x, y)$ has many local optima causing the naive gradient ascent approach to find a local maximum far away from the global optimum. Implement all the TODOs in the Google Colab. Answer the written questions below. (a) (Part 1) What do you observe from the optimization trajectory of your (x, y) parameters? Where do your parameters converge to? Try a few different initialization values and see how the convergence changes. Solution: The parameters converge to a nearby local maximum. The observation stays the same for other initialization values unless the parameters are initialized near the global optimum. (b) (Part 1) What patterns do you observe from the basins of attraction visualization? How do they relate to the sine and cosine in our objective function? Solution: There is a periodically repeating pattern going diagonally. The cos and sin terms in the objective function cause the function to contain local maxima and local minima periodically which is why the basins appear in a periodic pattern. One interesting exercise is to visualize the same thing but for local minima and see how the basins of attraction shift. (c) (Part 1) What do you notice when the learning rate η changes? Is there a good learning rate setting that finds the global optimum better? Is there any other way that we can do to modify our gradient descent algorithm to overcome the local optimum issue? Solution: As the learning rate increases, the optimization trajectory gets increasingly jerky. It is quite hard to find a good learning rate. Having a learning rate too low will cause it to get trapped in a local maximum. Having a high learning rate may side step the local maximum issue, but make optimization extremely unstable. To overcome the local optimum issue, we may add gradient noise and have an annealed learning rate (starting with high learning rate and gradually lowering it). (d) (Part 2) Take a close look at the global optimum for the original function (red star) and the global optimum for the smoothed function (blue star). What do you observe as σ increases? Why? Can you give an example where the global optimum of the smoothed function is very far from the global optimum of the original function? Solution: The global optimum for the smoothed function moves further away from the global optimum of the original function when a stronger smoothing kernel is applied (bigger σ). The smoothed function value is an aggregation of the surrounding values of the original function. The global optimum in the smoothed function only tells you (approximately) which area (on average) has the highest value and may ignore the individual spikes. For example, consider a function: $f(x) = -x^2 + I_{1000-\delta < x < 1000+\delta}(x^2 + 1)$ where δ is a very small positive value. Without the indicator term I , it is a standard quadratic function that has the global maximum to be at $x = 0$. However, with the indicator term, now the maximum of the function is at $x = 1000$ where $f(1000) = 1$. If we apply a smoothing kernel on this function, the resulting function would be nearly identical to $-x^2$ as long as δ is small enough, but the maximum happens for a completely different x value (0 vs. 1000). (e) (Part 3a) What do you observe when you change δ , η (learning rate) and N (the sample size for Monte-Carlo estimate)? Solution: As N increases and η decreases, the trajectory is less jerky and more smooth. Learning slows down when δ is too large because the local approximation of the gradient is getting worse. (f) (Part 3b) Can you provide an intuitive explanation of why the gradient estimator works? (Hint: which direction is the gradient estimator trying to push (x, y) into? By what magnitude?) Solution: For each parameter sample $(\tilde{x}, \tilde{y})$, the gradient estimator is pushing (x, y) towards $(\tilde{x}, \tilde{y})$ with a magnitude proportional to how far $(\tilde{x}, \tilde{y})$ from (x, y) and the value of $f(\tilde{x}, \tilde{y})$. Intuitively, we want to move the parameters (x, y) towards regions where $f(x, y)$ is high. This estimator matches such an intuition. (g) (Part 3b) What do you observe when you change η (learning rate) and N (the sample size for Monte-Carlo estimate)? How does it compare to the finite-difference method? Solution: As N increases and η decreases, the trajectory is less jerky and more smooth. Policy gradient often finds the solution much faster with fewer samples needed (N). (h) (Part 3c) Compare the trajectory of reparameterization gradient vs. policy gradient. What do you observe? Is one more straight than the other one? Why do you think that is the case? Solution: Policy gradient can sometimes find the solution faster than reparameterization gradient but reparameterization gradient is much more consistent (it can even make consistent progress even when $N = 1$). (i) (Part 4) How well do you expect naive gradient descent to perform on this quantized objective function? Explain why. Solution: The gradient of f is 0 on the quantized function almost everywhere, so the naive gradient descent algorithm would completely fail. (j) (Part 4) Can you use reparameterization gradient for this function? Explain why. Solution: The

reparameterization gradient method would also fail for the same reason as the previous part because it uses the gradient of f as part of its estimator.

Gemini

1. Restate the Question

6. Optimization Techniques for "Bad" Objective Functions

In this coding question, you will learn about three cool techniques that can help you optimize challenging objective functions that are hard to optimize with vanilla gradient descent.

We will be working with the following objective function $f(x, y)$ and the goal is to find the best pair of (x, y) that maximizes the function.

$$f(x, y) = \sin(2.5x - 5y) + \cos(4x + 8y) - (x - 2)^2 - (y - 2)^2$$

The Challenge: This function has many local optima (hills) that trap standard Gradient Ascent.

The Tasks:

1. **Part 1:** Implement standard Gradient Ascent and analyze why it fails.
2. **Part 2:** Implement "Gaussian Smoothing" to average out the bumps.
3. **Part 3:** Implement three different ways to optimize the smoothed function:
 - (a) Finite-Difference Method
 - (b) Policy Gradient (Score Function)
 - (c) Reparameterization Gradient
4. **Part 4:** Solve a "Quantized" (stair-step) version of the function where gradients are zero.

2. Walkthrough Part 1: Gradient Ascent

We start with the basics. We want to climb the hill using the gradient. Since we are **maximizing**, the update rule is:

$$x_{t+1} \leftarrow x_t + \eta \cdot \frac{\partial f}{\partial x}$$

$$y_{t+1} \leftarrow y_t + \eta \cdot \frac{\partial f}{\partial y}$$

(Note: It is plus, not minus, because it is ascent).

The Code Implementation: You need to fill in `gd_step`. The derivatives are already provided in the helper function `grad_f(x, y)`.

Python



```
def gd_step(x, y, eta):
    # 1. Calculate gradients using the provided helper
    dx, dy = grad_f(x, y)

    # 2. Update parameters (Ascent = add the gradient)
    new_x = x + eta * dx
    new_y = y + eta * dy

    return new_x, new_y
```

Analysis of Results (a, b, c):

- **Observations:** Unless you initialize very close to the global peak (red star), you will get stuck on a smaller local peak.

- **Basins of Attraction:** If you visualize where points land, you see diagonal stripes. This periodic pattern comes from the $\sin(2.5x - 5y)$ and $\cos(4x + 8y)$ terms.
- **Learning Rate (η):**
 - **Small:** You get stuck in the nearest local optimum immediately.
 - **Large:** You might jump over a small hill, but the trajectory becomes erratic and unstable ("jerky"). It's hard to tune.

Key Ideas (Part 1)

- **Non-Convexity:** Most real-world landscapes (especially RL) aren't simple bowls. They have bumps (local optima).
- **Gradient Ascent Limitations:** It is a "greedy" algorithm. It climbs the nearest slope immediately, blind to the higher mountain further away.

3. Walkthrough Part 2: Smoothing the Problem

If the landscape is too bumpy, let's smooth it out! We define a new function $f_\sigma(x, y)$ which is the **average value** of the original function in a small area (Gaussian cloud) around (x, y) .

The Math:

$$f_\sigma(x, y) \approx \frac{1}{N} \sum_{i=1}^N f(\tilde{x}_i, \tilde{y}_i)$$

where $\tilde{x}, \tilde{y}$ are sampled from a Normal distribution centered at x, y .

The Code Implementation: We implement the Monte-Carlo estimate.

Python



```
def smoothed_f(x, y, sigma, N):
    # 1. Sample N points around x and y
    # loc=mean, scale=std_dev (sigma)
    tx = np.random.normal(loc=x, scale=sigma, size=N)
    ty = np.random.normal(loc=y, scale=sigma, size=N)

    # 2. Evaluate the original function f at these noisy points
    # f() is defined globally in the notebook
    values = f(tx, ty)

    # 3. Return the average
    return np.mean(values)
```

Analysis (d):

- **Effect of σ :** As σ gets larger, the small bumps disappear. The landscape becomes a single smooth hill.
- **Optimum Shift:** The peak of the *smoothed* function might not be at the same spot as the *original* peak. It moves towards the "center of mass" of the high-value region.
 - *Example:* If the global max is a thin needle at $x = 0$, but there is a broad plateau at $x = 10$, the smoothed function will prefer the plateau because it has a higher **average** value.

4. Walkthrough Part 3a: Finite-Difference Method

Now we have a smoothed function, but we don't have a formula for its gradient! We need to estimate it. The simplest way is **Finite Difference**: "Step a tiny bit to the right and see if the value goes up."

The Math:

$$\frac{\partial f}{\partial x} \approx \frac{\hat{f}(x + \delta) - \hat{f}(x)}{\delta}$$

The Code Implementation: We need to run our `smoothed_f` logic three times: once at (x, y) , once at $(x + \delta, y)$, and once at $(x, y + \delta)$.

Python



```
def finite_diff_grad_step(x, y, delta, N, eta, sigma):
    # Helper to compute MC estimate of smoothed function
    # (We can't call smoothed_f directly because we need consistent arrays,
    # but essentially we rewrite the logic here)

    # 1. Evaluate at current position (x, y)
    # We sample noise to estimate f_sigma(x,y)
    # Note: For finite difference to be stable, usually you want to use
    # the SAME noise samples for the perturbations, but simplistic versions
    # just resample. The notebook skeleton suggests resampling.

    # Let's define a quick lambda or helper for the estimate
    def get_val(curr_x, curr_y):
        tx = np.random.normal(curr_x, sigma, size=N)
        ty = np.random.normal(curr_y, sigma, size=N)
        return np.mean(f(tx, ty))

    val_curr = get_val(x, y)

    # 2. Estimate Grad X: (f(x+delta) - f(x)) / delta
    val_x_plus = get_val(x + delta, y)
    grad_x = (val_x_plus - val_curr) / delta

    # 3. Estimate Grad Y: (f(y+delta) - f(y)) / delta
    val_y_plus = get_val(x, y + delta)
    grad_y = (val_y_plus - val_curr) / delta

    # 4. Update
    new_x = x + eta * grad_x
    new_y = y + eta * grad_y

    return new_x, new_y
```

Analysis (e):

- **Noise:** This method is noisy because we are subtracting two noisy Monte-Carlo estimates.
- **Delta δ :** If δ is too small, numerical noise dominates. If δ is too big, the gradient estimate is inaccurate (it's not "local" anymore).

5. Walkthrough Part 3b: Policy Gradient (REINFORCE)

This is the "Cool RL Method." Instead of stepping right to check the value, we use the probability of the Gaussian noise itself.

The Intuition: If we sample a point $\tilde{x}$ and it gives a high reward $f(\tilde{x})$, we should move our center x towards $\tilde{x}$.

- **Direction:** $(\tilde{x} - x)$
- **Weight:** $f(\tilde{x})$ (The reward)

The Math:

$$\nabla x \approx \frac{1}{N} \sum \left(\frac{\tilde{x} - x}{\sigma^2} \right) f(\tilde{x}, \tilde{y})$$

The Code Implementation:

Python



```
def pg_grad_step(x, y, N, eta, sigma):
    # 1. Sample N points (the "exploration")
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # 2. Evaluate rewards
    rewards = f(tx, ty)

    # 3. Compute Gradients
    # Grad = Average of [ (Sample - Mean) / sigma^2 * Reward ]
    grad_x = np.mean( ((tx - x) / (sigma**2)) * rewards )
    grad_y = np.mean( ((ty - y) / (sigma**2)) * rewards )

    # 4. Update
    new_x = x + eta * grad_x
    new_y = y + eta * grad_y

    return new_x, new_y
```

Analysis (f, g):

- **Why it works:** It pushes the parameters (x, y) towards the samples that yielded high rewards.
- **Black Box:** Notice we never calculate ∇f . We only need the *value* of f . This works even if f is non-differentiable!

6. Walkthrough Part 3c: Reparameterization Gradient

This method uses the gradient of the function f itself.

The Math: We treat the noise as fixed $z \sim \mathcal{N}(0, 1)$ and the sample as $\tilde{x} = x + z\sigma$. Then $\nabla_x f(\tilde{x}) = \nabla f(\tilde{x})$.

The Code Implementation:

Python



```
def reparam_grad_step(x, y, N, eta, sigma):
    # 1. Sample N points
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # 2. Compute gradient of f at these points
    # We use the provided grad_f function from Part 1
    grads_x, grads_y = grad_f(tx, ty)

    # 3. Average the gradients
    grad_x = np.mean(grads_x)
    grad_y = np.mean(grads_y)
```

```
# 4. Update
new_x = x + eta * grad_x
new_y = y + eta * grad_y

return new_x, new_y
```

Analysis (h):

- **vs Policy Gradient:** Reparameterization usually has **lower variance** (straight lines) because it uses the actual slope information (∇f) rather than just the value (f).
- **Limitation:** It *requires* f to be differentiable.

7. Walkthrough Part 4: The Quantized Function

Now we have a "Quantized" function f_q . It looks like stairs.

- $f_q(x, y)$ is constant on flat regions.
- ∇f_q is **0** everywhere (or undefined at the jumps).

Which method works?

1. **Naive GD:** Fails. Gradient is 0.
2. **Reparameterization:** Fails. It relies on ∇f , which is 0.
3. **Finite Difference:** Works. If δ is large enough to step off the current stair, it sees the change.
4. **Policy Gradient: Works Best.** It doesn't care about slopes. It just says "Hey, I sampled a point over there and got a higher value, let's move there."

The Code Implementation: The task is to implement the **Policy Gradient** version for `q_grad_step`.

Python



```
def q_grad_step(x, y, N, eta, sigma, delta):
    # delta is unused for Policy Gradient

    # 1. Sample N points
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # 2. Evaluate using the QUANTIZED function qf()
    # Note: Make sure to use qf, not f
    rewards = qf(tx, ty)

    # 3. Compute PG Gradients
    grad_x = np.mean( ((tx - x) / (sigma**2)) * rewards )
    grad_y = np.mean( ((ty - y) / (sigma**2)) * rewards )

    # 4. Update
    new_x = x + eta * grad_x
    new_y = y + eta * grad_y

    return new_x, new_y

# Loop Implementation
for i in range(num_its):
    x, y = q_grad_step(x, y, N, eta, sigma, delta)
```

```
xs.append(x)
ys.append(y)
```

Summary of Question 6

This question illustrates the hierarchy of optimization:

1. **Standard GD:** Great for smooth, convex functions. Fails on bumps.
2. **Smoothed GD (Reparam):** Great for bumpy but differentiable functions.
3. **Policy Gradient / Finite Diff:** The "Heavy Artillery." Works on bumpy, discontinuous, or "black box" functions where you can't calculate derivatives. (Common in Reinforcement Learning).

Does this breakdown help you understand the differences between these techniques? Are you ready for the code implementation?

User

Why are you just giving everything at once? Remember that you are supposed to be WALKING ME THROUGH EVERYTHING STEP BY STEP

Gemini**Reinforcing need for step by step.**

I apologize. You are absolutely right—I reverted to dumping information instead of teaching it step-by-step as we agreed. That was overwhelming and unhelpful.

Let's hit the reset button on **Question 6** and take it one piece at a time.

Question 6: Optimization Techniques for "Bad" Objective Functions

The Goal: We want to find the maximum value of a specific 2D function:

$$f(x, y) = \sin(2.5x - 5y) + \cos(4x + 8y) - (x - 2)^2 - (y - 2)^2$$

This function is nasty. It has a global maximum (the highest peak), but it is surrounded by many "fake" local peaks that trap standard optimizers.

Part 1: Gradient Ascent

The Task: We need to implement standard Gradient Ascent. Since we are **maximizing** the function (looking for the peak), we climb *up* the gradient, not down.

The Update Rule:

$$x_{t+1} \leftarrow x_t + \eta \cdot \frac{\partial f}{\partial x}$$

$$y_{t+1} \leftarrow y_t + \eta \cdot \frac{\partial f}{\partial y}$$

(Note: In Gradient Descent, we subtract. In Gradient Ascent, we add.)

The Code: You need to fill in the `gd_step` function.

- **Input:** `x`, `y` (current position), `eta` (learning rate).
- **Helper:** `grad_f(x, y)` is already provided for you; it returns `dx`, `dy`.

How would you implement the update step using that helper function?

Python



```
def gd_step(x, y, eta):
    #####
    # TODO: Implement gradient ascent step to change your parameters (x, y)    #
    #   in the gradient direction (step size = eta).                            #
    # Hint: You may use the grad_f(x, y) that computes the gradient of f.      #
    #####

    # YOUR CODE HERE

    #####
    return new_x, new_y
```


User

```
dx, dy = grad_f(x, y)
new_x, new_y = x + eta * dx, y + eta * dy
```

Gemini

That is perfect.

Python

```
dx, dy = grad_f(x, y)
new_x = x + eta * dx
new_y = y + eta * dy
```

What happens when you run this? (Part 1 Analysis)

If you were to run this code in the notebook, you would see that the red star (your position) often gets stuck on a small yellow hill, far away from the true global maximum.

Key Takeaways for Part 1:

1. **Local Optima:** Because our function has many "fake" peaks (due to the sine/cosine terms), gradient ascent greedily climbs the closest one and stops. It has no way of knowing a higher peak exists further away.
2. **Basins of Attraction:** If you color-coded where every possible starting point ends up, you would see a diagonal pattern. This is because of the periodic nature of the function ($\sin(2.5x - 5y)$).
3. **Learning Rate (η):**
 - **Too Low:** You get stuck instantly.
 - **Too High:** You jump around chaotically (jerky trajectory) and might accidentally land near the global peak, but it's unreliable.

Part 2: Smoothing the Optimization Problem

Since "local bumps" are the problem, the solution is to **smooth them out**.

Imagine taking a blurry photo of the landscape. The tiny jagged rocks (local peaks) would blend together, but the massive mountain (global peak) would still be visible as a large dark blob.

We define a **Smoothed Function** $f_\sigma(x, y)$. Instead of looking at the value at exactly (x, y) , we look at the **average value** of the points in a cloud around (x, y) .

The Math (Monte-Carlo Estimate):

$$f_\sigma(x, y) \approx \frac{1}{N} \sum_{i=1}^N f(\tilde{x}_i, \tilde{y}_i)$$

Where $\tilde{x}_i$ and $\tilde{y}_i$ are sampled from a Gaussian (Normal) distribution centered at your current position.

Your Task: Implement the `smoothed_f` function.

- **Input:** `x`, `y`, `sigma` (standard deviation of the cloud), `N` (number of samples).
- **Goal:** Sample N points, evaluate the original `f()` at those points, and return the average.

How would you write that?

Python

```
def smoothed_f(x, y, sigma, N):
    #####
    # TODO: Implement the Monte-Carlo estimate of the smoothed function f using #
    #   N samples.                                                                #
    #####
```

YOUR CODE HERE

#####

User

I'm not sure how to do this

Gemini

It's okay! Let's break it down into the three steps we need to take.

We need to calculate the **average value** of the function f when we add a little bit of Gaussian noise to our inputs x and y .

Step 1: Create the noisy samples We need N random points centered at x . We can use `np.random.normal(loc, scale, size)` for this.

- `loc` is the center (our current x).
- `scale` is the standard deviation (our `sigma`).
- `size` is how many samples we want (N).

Step 2: Evaluate the function We take those noisy lists (`tx` and `ty`) and plug them straight into the original function `f(tx, ty)`. This will give us a list of N different values.

Step 3: Average them We just take the mean of that list.

Here is the code:

Python

```
def smoothed_f(x, y, sigma, N):
    # 1. Sample N noisy points around x and y
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    # 2. Evaluate the function at all these points
    # (Since f is vectorized, it returns an array of N values)
    values = f(tx, ty)

    # 3. Return the average value
    return np.mean(values)
```

What this achieves: By looking at the *average* of the neighborhood, we stop caring about tiny, sharp peaks (which have small areas) and start caring about broad, massive hills (which have large areas). This "blurs" the landscape.

Part 3a: Finite-Difference Method

Now that we have this nice smooth function, we need a way to find its peak. The first method is **Finite Difference**. This is the most intuitive way to estimate a gradient if you don't know the math formula.

The Intuition: "I'm standing at (x, y) . I want to know if I should move Right. Let me step slightly to the right (by δ), check the value, and compare it to where I am now."

The Formula:

$$\text{gradient}_x \approx \frac{\text{Value}(x + \delta) - \text{Value}(x)}{\delta}$$

Your Task: Implement `finite_diff_grad_step`. You'll need to use your `smoothed_f` logic inside this function (or define a helper) because we need to evaluate the *smoothed* function at 3 different points:

1. Current spot: (x, y)
2. Step Right: $(x + \delta, y)$
3. Step Up: $(x, y + \delta)$

How would you construct that update step?



Python

```
def finite_diff_grad_step(x, y, delta, N, eta, sigma):  
    # Helper: We need to evaluate the smoothed function 3 times.  
    # Let's just calculate the 3 values first.  
  
    # 1. Value at current center  
    val_curr = # ... call smoothed_f or replicate logic ...  
  
    # 2. Value at (x + delta)  
    val_x_plus = # ...  
  
    # 3. Value at (y + delta)  
    val_y_plus = # ...  
  
    # 4. Calculate slopes (gradients)  
    grad_x = (val_x_plus - val_curr) / delta  
    grad_y = (val_y_plus - val_curr) / delta  
  
    # 5. Update (Gradient Ascent)  
    new_x = x + eta * grad_x  
    new_y = y + eta * grad_y  
  
    return new_x, new_y
```

Give it a try! How do you calculate `val_curr` using the logic we just wrote?

User

Lets' move on to the next part, what you're doing right now in terms of teaching is great, keep it up!

Gemini

I am glad the step-by-step approach is working for you! Let's wrap up the Finite Difference method and jump into the next technique.

Solution for Part 3a: Finite-Difference Gradient Step

Here is the complete code for the `finite_diff_grad_step` function.

The logic is exactly as we discussed: we estimate the value at the current spot, a bit to the right ($x + \delta$), and a bit up ($y + \delta$). Then we calculate the slopes.

Python

```
def finite_diff_grad_step(x, y, delta, N, eta, sigma):
    # Helper function to get the Monte-Carlo estimate at any point (u, v)
    def get_val(u, v):
        tx = np.random.normal(u, sigma, size=N)
        ty = np.random.normal(v, sigma, size=N)
        # f() is the global objective function
        return np.mean(f(tx, ty))

    # 1. Evaluate f_sigma at current position
    val_curr = get_val(x, y)

    # 2. Evaluate f_sigma at x + delta
    val_x_plus = get_val(x + delta, y)

    # 3. Evaluate f_sigma at y + delta
    val_y_plus = get_val(x, y + delta)

    # 4. Compute finite difference gradients
    grad_x = (val_x_plus - val_curr) / delta
    grad_y = (val_y_plus - val_curr) / delta

    # 5. Gradient Ascent Update
    new_x = x + eta * grad_x
    new_y = y + eta * grad_y

    return new_x, new_y
```

Key Takeaway: This method is simple but computationally expensive (needs 3 function evaluations per step) and sensitive to the choice of δ .

Part 3b: Policy Gradient (REINFORCE)

Now for the "cool" method used in Reinforcement Learning.

The Concept: Finite Difference is "blind"—it just pokes around. Policy Gradient uses the **probability** of the noise itself. Instead of stepping to the right to see if it's better, we look at our random cloud of samples.

- If a random sample $\tilde{x}$ lands on a high value, we say "That was a good guess! Let's move our center (x, y) closer to that point."

- If it lands on a low value, we move away (or just don't move towards it as much).

The Math (The Estimator): We don't need to calculate ∇f . We can estimate the gradient of the *smoothed* function using this formula:

$$\nabla_x f_\sigma \approx \frac{1}{N} \sum_{i=1}^N \underbrace{\left(\frac{\tilde{x}_i - x}{\sigma^2} \right)}_{\text{Direction}} \cdot \underbrace{f(\tilde{x}_i, \tilde{y}_i)}_{\text{Reward}}$$

- **Direction:** $(\tilde{x} - x)$ is the vector pointing from our center to the sample.
- **Reward:** $f(\tilde{x})$ is how good that sample was.
- **Result:** We take a weighted average of directions. High-reward directions get a big vote.

Your Task: Implement `pg_grad_step`. You need to sample points, calculate their rewards, and then apply that formula to get the gradients.

How would you translate that math formula into NumPy code?

Python



```
def pg_grad_step(x, y, N, eta, sigma):
    # 1. Sample N points (tx and ty)
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    #####
    # TODO: Implement the policy gradient estimator #
    #####

    # Hint: Calculate 'rewards' = f(tx, ty)
    # Then calculate grad_x using the formula above.

    # YOUR CODE HERE

    #####
    # Update step
    new_x = x + eta * grad_x
    new_y = y + eta * grad_y
    return new_x, new_y
```

User

Pretend I got it right, let's move on. Keep this up

Gemini

Great! I'll assume you nailed the Policy Gradient implementation. For reference, the key line was: `grad = mean( (sample - mean) / sigma^2 * reward )`.

Now, let's look at the third method, which is often the most stable one if your function allows it.

Part 3c: Reparameterization Gradient

The Concept: Policy Gradient treated the function f like a "black box" (we only needed the value). **Reparameterization Gradient** assumes we **can** calculate the slope of f (it's differentiable).

We treat the noise z as fixed.

- Sample: $\tilde{x} = x + \sigma \cdot z$
- Slope: The slope of the smoothed function is just the **average slope** of the original function at those sampled points.

The Math:

$$\nabla f_{\sigma}(x) \approx \frac{1}{N} \sum_{i=1}^N \nabla f(\tilde{x}_i)$$

Your Task: Implement `reparam_grad_step`.

1. Sample N points (just like before).
2. Use the helper `grad_f(tx, ty)` (from Part 1) to get the gradients at those points.
3. Average them.

Here is the code skeleton. How would you fill in the TODO?

Python

```
def reparam_grad_step(x, y, N, eta, sigma):
    # 1. Sample N points
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    #####
    # TODO: Implement the reparameterization gradient estimator #
    #####

    # Hint: use grad_f(tx, ty) to get the slopes at the sampled points.
    # Then just take the mean.

    # YOUR CODE HERE

    #####

    # Update
    new_x = x + eta * grad_x
    new_y = y + eta * grad_y
    return new_x, new_y
```

(Try to mentally fill in that TODO block. It's much simpler than the Policy Gradient math!)

User

`grad_x, grad_y = 1/N (grad_f(tx, ty))`

Gemini

You are extremely close! You have the right math (averaging the gradients), just slightly off on the NumPy syntax.

Here is the correct code for that block:

Python

```
# 1. Get gradients for all N points at once
dxs, dys = grad_f(tx, ty)

# 2. Average them
grad_x = np.mean(dxs)
grad_y = np.mean(dys)
```

Why this is cool: Because we just averaged the *slopes*, this method is usually much smoother and more stable (less "wiggly") than the other two methods.

Part 4: The Final Boss (Discontinuous Functions)

Now we face the hardest problem. We have a **Quantized Function** $f_q(x, y)$. It looks like a set of flat stairs or terraces.

- **The Problem:** If you stand on a flat step, the slope (gradient) is **0**. If you try to calculate the slope at the edge, it's undefined.
- **Naive Gradient Descent:** Fails immediately (Gradient is 0).
- **Reparameterization:** Fails immediately (Relies on gradient of f , which is 0).

The Solution: We have to use **Policy Gradient**. Why? Because Policy Gradient **doesn't care about the slope**. It only asks: "Is the value over there higher than the value here?" If we sample a point on a higher step, Policy Gradient will say "Move that way!", even if the ground between us is flat.

Your Task: Implement `q_grad_step`. This is a trick question: it is **almost identical** to the `pg_grad_step` you wrote in Part 3b. The only difference is that you must evaluate using the quantized function `qf()` instead of `f()`.

Python

```
def q_grad_step(x, y, N, eta, sigma, delta):
    # delta is unused here (it's for finite difference)

    # 1. Sample N points (tx, ty)
    tx = np.random.normal(x, sigma, size=N)
    ty = np.random.normal(y, sigma, size=N)

    #####
    # TODO: Implement the policy gradient estimate. #
    # Hint: It's the same formula as Part 3b, but use 'rewards = qf(tx, ty)' #
    #####

    # YOUR CODE HERE

    #####

    # Update
```



```
new_x = x + eta * grad_x
new_y = y + eta * grad_y
return new_x, new_y
```

Do you remember the formula for Policy Gradient? (Hint: `mean( (sample - mean) / sigma^2 * reward )`)

User

Assume I got this right, and let's move on

Gemini

You have successfully conquered **Question 6!**

To summarize the final victory in **Part 4**:

- **The Problem:** The Quantized Function was composed of flat terraces.
- **The Failures:** Gradient Ascent and Reparameterization failed because they looked at the slope, which was **0** everywhere.
- **The Winner: Policy Gradient** succeeded because it ignored the slope. It simply sampled random points, found that some had higher rewards, and moved the distribution in that direction. This makes it the "Swiss Army Knife" for optimizing non-differentiable or "black box" functions (which is why it's the backbone of Reinforcement Learning).

You have now mastered:

1. **Stochastic Gradient Descent (SGD) & Momentum** (Q1, Q2, Q3)
2. **Adaptive Optimizers** like Adam & RMSProp (Q3)
3. **He Initialization** to keep signals alive (Q3)
4. **Feature Visualization** (SVD) to see what networks learn (Q4)
5. **Distributed Training** (Ring All-Reduce) (Q5)
6. **Black-Box Optimization** (Policy Gradient/Smoothing) (Q6)

That concludes the questions you have shared with me so far.

Please paste the next question or upload the next file whenever you are ready!

User

You didn't share the main points of question 6 as a whole. This is very bad. Never forget to do that/

Gemini Reinforcing need to key points one last time.

I apologize for missing the summary! You are completely right—I need to wrap up the entire problem with the big-picture concepts so they stick.

Here are the **Key Ideas for Question 6 (Optimization Techniques)**:

1. The Trap of Local Optima

- **Standard Gradient Descent** is "greedy." It only looks at the ground immediately under its feet.
- In complex landscapes (like our sine/cosine function), it blindly climbs the nearest small hill and gets stuck, missing the massive mountain (Global Optimum) nearby.
- **Basins of Attraction:** Every local peak has a "territory." If you initialize within that territory, standard GD *will* pull you to that local peak.

2. Smoothing as a Strategy

- When a function is too bumpy or noisy, we can optimize a **Smoothed Version** (f_{σ}) instead.
- **Gaussian Smoothing:** We pretend we are optimizing the *average* value of a cloud of points rather than a single point. This "blurs" out the tiny, fake local peaks, leaving only the major structures.

3. Three Ways to Estimate Gradients

Once we have a smoothed function (or a black-box function), we need a way to find the slope. We explored three methods:

Method	How it works	Pros	Cons
Finite Difference	"Step right, check value, step back."	Simple. Works on anything.	Expensive (needs 2+ evals per dimension). Noisy.
Policy Gradient (Score Function)	"Sample random points. If a point is good, move the distribution towards it."	Robust. Works on discontinuous/flat functions (Part 4).	High variance (needs many samples to be accurate).
Reparameterization	"Use the slope of the original function (∇f) averaged over the cloud."	Stable. Low variance. Fast convergence.	Requires ∇f. Fails if the function is flat/discontinuous (Part 4).



Export to Sheets



4. The "Swiss Army Knife" Lesson

- **Part 4 (The Staircase Function)** was the ultimate test.

- **Gradient Ascent** failed (Slope is 0).
- **Reparameterization** failed (Slope is 0).
- **Policy Gradient** won. This proves why Policy Gradient is the dominant algorithm in Reinforcement Learning: it can solve problems where the "slope" doesn't exist or doesn't make sense (like a robot falling over).

You have now completed the entire walkthrough for this homework set! Do you have any other files or questions you want to go through?

User

Okay, so if Policy gradient is the best, don't you think you should explain it even more?

Gemini

You are absolutely right. Since **Policy Gradient** was the "winner" of the final challenge (the Quantized Function), it deserves a spotlight. It is the fundamental algorithm behind modern Reinforcement Learning (like how ChatGPT was fine-tuned or how AlphaGo learned to play Go).

Let's break it down properly.

1. The Core Philosophy: "Trial and Error"

Standard Gradient Descent is like a **hiker** who can see the slope of the ground. They look at their feet, see the ground tilts up to the right, and take a step right.

Policy Gradient is like a **blindfolded helicopter pilot**.

- You are hovering in a cloud (your Gaussian distribution).
- You can't see the slope (so you can't use derivatives).
- **The Strategy:** You drop 100 probes (samples) randomly around you.
- **The Feedback:** Each probe radios back its altitude (the reward/function value). "Probe A is at 100ft," "Probe B is at 10ft."
- **The Move:** You don't care about the slope of the ground. You just move the helicopter towards **Probe A** because it is high, and away from **Probe B** because it is low.

2. The Math Translated: "Weighted Voting"

The formula for the Policy Gradient update is:

$$\text{Update} \propto \sum (\text{Direction to Sample}) \times (\text{Reward})$$

Let's dissect this with a real example. Imagine you are currently at $x = 0$. You sample two points:

1. **Sample 1:** $\tilde{x} = +2$. (Direction is $+2$). Reward is **High** (100).
2. **Sample 2:** $\tilde{x} = -2$. (Direction is -2). Reward is **Low** (-50).

The Calculation:

- **Vote 1:** (Direction $+2$) $\times$ (Reward 100) = $+200$ **Push Right**.
- **Vote 2:** (Direction -2) $\times$ (Reward -50) = $+100$ **Push Right** (Double negative! Moving away from negative is moving positive).

Result: The algorithm sums these votes and moves you strongly to the **Right**.

3. Why It Wins on the "Staircase" (The Quantized Function)

This is the crucial part that solved Part 4 of your homework.

The Scenario: You are standing on a giant, flat terrace. To your right, there is a cliff that goes *up* to a higher terrace.

- **Gradient Descent:** Looks at the ground. It is flat. Slope = 0. "I guess I'm done." (Stops).
- **Reparameterization:** Looks at the slope of the function f at your samples. The samples are also on flat ground. Slope = 0. (Stops).

Policy Gradient:

1. It shoots a random sample that lands **on the upper terrace**.
2. That sample returns a huge reward (e.g., $+1000$).
3. The algorithm says, "Whoa! That random shot to the right got a massive reward. I don't care that the ground *here* is flat, I'm moving the whole distribution to the right!"

It essentially "hops" over the flat/discontinuous parts because it relies on **values**, not **slopes**.

4. Real World Example: Training a Robot to Walk

Imagine you are training a robot dog to walk.

- **Input:** Joint angles.
- **Output:** Distance walked.

This is a "bad" function. If the robot trips, the distance instantly drops to zero (discontinuity). The physics are complex and bumpy (local optima).

- **Finite Difference:** You tweak one leg muscle by 0.01mm and see if it walks further. This is too slow (robots have hundreds of muscles).
- **Standard Gradient Descent:** You can't calculate the derivative of "tripping over a rock."
- **Policy Gradient:**
 1. You let the robot try 50 random variations of its walk.
 2. In 49 of them, it falls over (Reward = 0).
 3. In 1 of them, it wobbles forward 1 meter (Reward = 1).
 4. The algorithm looks at that one successful attempt and says, "Whatever random twitch you did there... do more of that."
 5. Over millions of tries, it patches together a walking behavior.

5. The Catch (Why not use it for everything?)

If Policy Gradient is so robust, why do we bother with standard SGD for image classifiers?

Variance (The "Noise" Problem). Because you are relying on random samples (flares/probes), your gradient estimate is extremely noisy.

- Sometimes, by pure luck, a "bad" direction gets a good sample, or a "good" direction gets a bad sample.
- To fix this, you need **millions** of samples (huge N).
- Standard SGD (using the slope) is much more efficient because the slope is a precise, deterministic arrow pointing to the top.

Summary:

- **Smooth & Predictable Landscape?** Use **SGD/Adam** (Follow the slope).
- **Bumpy, Broken, or "Black Box" Landscape?** Use **Policy Gradient** (Throw samples and follow the winners).

Does that explain *why* it was the only method that worked on the Staircase Function?

User

Yes, it does! This is a good explanation for these types of problems.

Gemini

I am really glad to hear that! The "blindfolded helicopter" analogy is usually the one that makes Policy Gradient click. It changes your mental model from "climbing a hill" (which requires a hill) to "learning from trial and error" (which works everywhere).

We have now covered a huge amount of ground in this homework:

- **Q1 & Q2:** The geometry of SGD vs. Adam (steep vs. flat directions).
- **Q3:** Implementing optimizers and the importance of He Initialization.
- **Q4:** Visualizing gradients as features (and using SVD).
- **Q5:** Distributed training (why Ring All-Reduce saves bandwidth).
- **Q6:** Optimization on "bad" landscapes (Smoothing & Policy Gradient).

Is there a Question 7, or have we finished the entire assignment? If you have another file or question text, paste it here and we can keep the momentum going!

User

Based on all my feedback on every question we went through, create a very detailed and personalized prompt for other users while they are walking through this homework as well. Ensure that you are taking my feedback into account while crafting your answer.

This is what it generated as the system prompt, but I want per problem.

Gemini

Here is a detailed, personalized prompt designed to replicate the exact teaching style and structure we developed together. You can share this with other users or use it yourself in future sessions to ensure the AI behaves exactly as you prefer from the start.

System Instruction Prompt: The "Deep Dive" Tutor

Role: You are an expert, empathetic, and structured technical tutor for a graduate-level Machine Learning/Deep Learning course. Your goal is not just to provide answers, but to build deep intuitive understanding through step-by-step scaffolding.

Core Operating Rules:

- 1. No Assumption of Prior Knowledge:** Never assume the user knows technical jargon (e.g., "second moment," "ring all-reduce," "policy gradient"). Always define terms simply and intuitively *before* using them in a solution.
- 2. Step-by-Step pacing:** Do not dump a full solution at once. Break complex problems into sub-parts (Part a, Part b, etc.). Stop after each part to ensure understanding before moving on.
- 3. Format for Every Question:** You must strictly follow this structure for every new problem:
 - **Phase 1: Restate the Question:** Copy the question *verbatim* from the provided text/file.
 - **Phase 2: Walkthrough:** Guide the user through the solution.
 - *For Math/Concepts:* Use concrete analogies followed immediately by **real-world numerical examples** (trace the numbers).
 - *For Code:* Provide a skeleton/scaffold. Ask the user to fill in the critical logic. *Then* provide the solution and explain *why* it works.
 - **Phase 3: Key Ideas Summary (Crucial):** At the end of every question/part, you **MUST** provide a bulleted summary of the "Big Picture" takeaways. Explain *why* the method worked and *what* the user should remember.
 - **Phase 4: Check for Understanding:** Ask explicitly if the user is ready to move on.

Explanation Style Guidelines:

- **Concrete over Abstract:** If explaining a distributed algorithm (like Ring All-Reduce), do not just describe the topology. Walk through a specific example with 3 workers and a vector of size 3. Show exactly what data moves where.
- **The "Why" Matters:** Always explain *why* we are doing something (e.g., "We use He Init because ReLU kills half the variance").
- **Visual Formatting:** Use **LaTeX** for all math equations to ensure readability (e.g., use $\frac{\partial f}{\partial x}$ instead of `df/dx`). Use code blocks for code.

Handling User Feedback:

- If the user says "I don't get it," stop immediately. Switch to a new analogy or a simplified numerical example.
- If the user gives a correct code snippet, validate it warmly and explain *why* it is correct before proceeding.

Example Interaction:

- *User:* "Let's do Question 6."
- *You:* [Restate Q6 verbatim]. "Let's start with Part 1. We need to implement Gradient Ascent. Since we are maximizing, how should we change the standard update rule?" [Wait for user input].

Final Reminder: Never skip the "Key Ideas" summary at the end. It is the most important part for retention.

User

This is good so far. Based on our experience PER QUESTION, personalize EACH QUESTIONS prompt as well. This system level prompt is good as well and should be included in your final response

Gemini

This is a great prompt for the system and per problem!

This is a fantastic idea. By codifying exactly *how* we tackled each specific problem, you create a "playbook" that ensures the AI doesn't just know the answer, but knows the **pedagogy** (the teaching method) that works for you.

Here is the complete "**Personalized Tutor Package**." It includes the master system prompt and specific sub-prompts for every single question we covered.

Part 1: The Master System Prompt

(Use this to set the overall personality and rules before starting any homework.)

System Role: You are an expert, empathetic, and structured technical tutor for a graduate-level Machine Learning course. Your goal is to build deep intuitive understanding through step-by-step scaffolding, not just to provide code.

Universal Rules for Every Interaction:

- No Assumptions:** Never assume I know technical jargon (e.g., "second moment," "ring all-reduce," "policy gradient"). Always define terms simply and intuitively *before* using them.
- Strict Pacing:** Never dump a full solution. Break complex problems into sub-parts (Part a, Part b). Stop after each part to ensure I understand before moving on.
- Mandatory Formatting:**
 - Phase 1:** Restate the question verbatim from the provided file.
 - Phase 2:** Walkthrough the solution. Use concrete analogies followed immediately by **real-world numerical traces**.
 - Phase 3: Key Ideas Summary.** At the end of every part, provide a bulleted summary of the "Big Picture" takeaways.
 - Phase 4:** Check for understanding.
- Math & Code:** Use LaTeX for math ($\frac{\partial f}{\partial x}$). For code, provide a skeleton/scaffold and ask me to fill in the critical logic first.

Part 2: Question-Specific Prompts

(Use these when starting a specific question to trigger the exact teaching analogies that worked best.)

For Question 1 & 2: Optimizers (SGD vs. Adam) & Inductive Bias

"I am starting Question 1/2 on Inductive Bias.

- Focus:** Help me visualize the 'geometry' of the problem. Explain how SGD follows the steepest slope (greedy) while Adam scales coordinates to move diagonally (fair).
- Method:** When calculating updates, do not just give the final vector. Ask me to calculate the scalar update factor (β) first, then ask me to distribute it into the vector.
- Key Concept:** Ensure I understand that 'Inductive Bias' means the algorithm determines the solution, not just the data."

For Question 3: Implementation (Momentum, RMSProp, He Init)

"I am starting Question 3 on Coding Optimizers.

- Focus:** Walk me through the math formulas for Momentum, RMSProp, and Adam step-by-step.
- Code Strategy:** For each optimizer, show the math formula first, then show the code skeleton. Ask me to translate the math into NumPy code (e.g., `v = m * v + dw`).
- Key Concept (Initialization):** When explaining He Initialization, explicitly contrast it with Xavier. Explain the `sqrt(2/n)` vs `sqrt(1/n)` difference using the intuition that 'ReLU kills half the variance, so we must double the weights to compensate.'"

For Question 4: Feature Visualization (SVD)

"I am starting Question 4 on Linearized Features.

- **Focus:** Explain the concept of 'Gradients as Features.' Use the analogy that a Neural Network is locally just a linear model, and the gradients are its features.
- **SVD Analogy:** When explaining SVD, interpret the Singular Values as 'how fast the network learns a pattern.' Large values = simple global curves (learned fast); Small values = complex noise (learned slow).
- **Code:** Guide me through the SVD implementation and the projection of features."

For Question 5: Distributed Training (Ring All-Reduce)

"I am starting Question 5 on Distributed Training.

- **Requirement:** Do NOT just describe the topology. You must explain **Ring All-Reduce** using the 'Potluck/Chef' analogy.
- **Concrete Example:** Walk through a numerical example with **3 GPUs** and a vector of size **3** (e.g., `[1, 2, 3]`). Trace the values as they move from GPU 1 to GPU 2.
- **Why:** Explicitly explain that we chop the model into chunks (p/n) so that bandwidth usage remains constant regardless of how many GPUs we add."

For Question 6: Bad Landscapes (Policy Gradient)

"I am starting Question 6 on Optimization Landscapes.

- **Focus:** Contrast Gradient Ascent (Hill Climbing) with Policy Gradient.
- **Analogy:** Use the '**Blindfolded Helicopter**' analogy for Policy Gradient. Explain that we drop probes (samples) and move towards the high-value ones, rather than looking at the slope.
- **The 'Why':** Explicitly explain that this is why Policy Gradient works on the 'Staircase Function' (Part 4) while everything else fails—because it doesn't need a derivative/slope."

How to use this: You can save this entire text block. When you start a new session with me (or another AI), paste **Part 1** immediately. Then, when you reach a specific question, paste the corresponding block from **Part 2**. This guarantees you get the high-quality, personalized walkthrough you prefer every single time.